

เอกสารระบบสำหรับผู้ดูแล

สุขฤทัย ฟิตเนส · ระบบสมาชิกและเช็คอิน · <https://srv1979069.hstgr.cloud>

รุ่นเอกสาร **1.3** | 20 กันยายน 2569 | เขียนจากรุ่น **b8a9d96** ที่ผ่านการตรวจแล้วและกำลังจะขึ้นเครื่องจริง (รุ่นที่ให้บริการอยู่ขณะนี้คือ **856846c**)

เอกสารนี้ไม่ใช่คู่มือพนักงาน — คู่มือฉบับพนักงาน (เล่มสีส้ม 2.7) ว่าด้วยการใช้งานหน้าจอ ส่วนเล่มนี้ว่าด้วย **เครื่อง** **ข้อมูล** และการดูแลระบบ สำหรับเจ้าของยิมและผู้ดูแลเครื่อง

ไม่มีรหัสผ่าน กุญแจ หรือคีย์ใด ๆ ในเล่มนี้ ทุกที่ที่ต้องอ้างถึงคีย์จะเขียนเป็นชื่อตัวแปรเท่านั้น ถ้าพบว่ามีค่าจริงหลุดอยู่ในสำเนาฉบับใด ให้ถือว่าสำเนานั้นต้องถูกทำลายและออกใหม่

เล่มนี้เขียนโดยใคร บทที่ 1 และ 2 โดย Project Planner · บทที่ 3 โดย Infrastructure Engineer (ตรวจจากเครื่องจริง 16 กันยายน 2569) · บทที่ 4 โดย Full Stack Developer (ตรวจจากโค้ดจริง) · รวมเล่ม ตรวจความสอดคล้อง และเพิ่มภาคผนวกโดย Project Planner

สารบัญ

บท	เรื่อง	ใครควรอ่าน
1	อ่านหน้าเดียวจบ (สำหรับเจ้าของยิม) — ข้อมูลอยู่ที่ไหน ใครเข้าถึงได้ ระบบทำอะไรให้เอง เครื่องพังทำอย่างไร งานประจำ	เจ้าของยิม
2	ข้อมูลส่วนบุคคลของลูกค้า — 14 ประเภทข้อมูล ที่เก็บ ผู้เข้าถึง อายุการเก็บ การลบ · "เก็บ 180 วัน" นับสามชั้น · ข้อมูลที่ออกนอกเครื่อง	เจ้าของยิม · ผู้ดูแล
3	โครงสร้างพื้นฐาน — เครื่องและ OS · Docker/Caddy/HTTPS · โฟลเดอร์และ volume · .env · สำรอง/กู้คืน/งานตามเวลา · พอร์ตและไฟร์วอลล์ · อัปเดตรุ่น · ย้ายเครื่อง · งานประจำ · คำสั่งที่ใช้อยู่	ผู้ดูแลเครื่อง
4	แอปพลิเคชัน — สดัก · repo · ฐานข้อมูลและ migration · ที่เก็บไฟล์ · สิทธิ์และ endpoint · การเข้ารหัสและ token · อีเมล · รายงานของเจ้าของยิมและ migration 018 · เทสต์ · รันบนเครื่องพัฒนา	นักพัฒนา
ก	ภาคผนวก — คำศัพท์ · รายการตรวจรายสัปดาห์/เดือน/ไตรมาส · ใครติดต่อเรื่องอะไร · ขอบเขตของเอกสาร	ทุกคน

อ่านเท่าที่จำเป็น เจ้าของยิมที่ไม่ได้ดูแลเครื่องเอง อ่านบทที่ 1 และ 2 จบก็พอ · คนที่รับช่วงดูแลเครื่องต่อจากทีม ให้เริ่มที่บทที่ 3 แล้วเก็บบทที่ 4 ไว้เปิดเมื่อต้องแก้ระบบ

บทที่ 1 · อ่านหน้าเดียวบ (สำหรับเจ้าของยม)

หน้านี้เขียนให้คนที่ไม่ได้ทำงานสายคอมพิวเตอร์อ่านจบได้ในสองนาที บทถัดไปเป็นรายละเอียดสำหรับคนที่ต้องลงมือดูแลเครื่อง

ข้อมูลของลูกค้าอยู่ที่ไหน

อยู่บน เครื่องเซิร์ฟเวอร์ของยิมเครื่องเดียว ที่เช่าไว้ (VPS ของ Hostinger) ทั้งหมด — ชื่อ เบอร์ อีเมล รูปถ่าย ประวัติเช็คอิน และประวัติการรับเงิน

ไม่มีข้อมูลลูกค้าถูกส่งออกไปเก็บที่บริษัทอื่น ไม่มีบัญชีคลาวด์ของเจ้าอื่นมาเกี่ยวข้อง ระบบส่งข้อมูลออกจากเครื่องอยู่อย่างเดียวคือ อีเมลที่ส่งถึงลูกค้าและพนักงาน ซึ่งออกผ่านกล่องจดหมาย Microsoft 365 ของยิมเอง

ใครเปิดดูได้บ้าง

ใคร	เห็นอะไร
พนักงาน (บัญชีที่คุณสร้างให้)	รายชื่อสมาชิก บัตร การเช็คอิน การรับเงิน และแก้ไขข้อมูลสมาชิกได้
เจ้าของยิม	ทุกอย่างของพนักงาน + ตั้งค้ายิม แพ็กเกจ บัญชีผู้ใช้ อีเมลของระบบ และภาพหน้าจอในเรื่องที่พนักงานแจ้ง
สมาชิก	เฉพาะส่วน "ช่วยเล่น" ของตัวเอง — วิธีใช้เครื่องและโปรแกรมฝึก ไม่เห็นข้อมูลของสมาชิกคนอื่นเลย
คนที่ไม่ได้ล็อกอิน	เปิดได้เฉพาะหน้าวิธีใช้เครื่องที่ผูกกับ QR ติดเครื่อง ซึ่งไม่มีข้อมูลบุคคลอยู่เลย

ทีมพัฒนาไม่มีบัญชีถาวรในระบบของคุณ การเข้าเครื่องเพื่อแก้ปัญหาหรืออัปเดตรุ่นทำผ่านช่องทางของผู้ดูแลเครื่อง ซึ่งคุณเป็นคนถือสิทธิ์

เรื่องที่ระบบทำให้เองทุกวัน

- **สำรองข้อมูลอัตโนมัติทุกคืนเวลา 02:17** เก็บไว้บนเครื่องเดียวกัน ย้อนหลัง **7 วัน** · ระหว่างนั้นระบบหยุดสั้น ๆ ไม่กี่วินาที ถ้ามีคนเปิดหน้าเว็บพอดีอาจเจอข้อผิดพลาดชั่วขณะแล้วกลับมาเอง — เลือกเวลานี้เพราะยิมปิดแล้ว
- **บันทึกว่าใครทำอะไร** — การแก้ไขข้อมูลสมาชิก การเปิดดูภาพหน้าจอที่พนักงานแจ้ง การออกบัตรใหม่ ระบบเขียนไว้ทุกครั้งว่าใครทำและทำเมื่อไหร่
- **รหัสผ่านทุกบัญชีถูกเข้ารหัสไว้** ไม่มีหน้าจอไหนแสดงรหัสของใครกลับมาได้ รวมทั้งรหัสกล่องจดหมายของยิม

ถ้าเครื่องพัง จะเป็นอย่างไร

ชุดสำรองอยู่บนเครื่องเดียวกัน ดังนั้น เครื่องพังพร้อมกันทั้งเครื่องคือกรณีที่ยังกู้ไม่ได้ ถ้าไม่มีสำเนาอยู่ที่อื่น — เป็นข้อจำกัดที่ต้องรู้ ไม่ใช่ความลับ

ทางที่เลือกไว้แล้ว: ก๊อปปี้ชุดสำรองออกนอกเครื่องสัปดาห์ละครั้ง ด้วยคำสั่งเดียว (scp) ลงคอมพิวเตอร์ของยิม — ไม่มีค่าใช้จ่ายเพิ่ม ใช้เวลาไม่กี่นาที ขั้นตอนอยู่ในข้อ 3.5 · **สแนปช็อตของผู้ให้บริการ VPS** เป็นทางเลือกที่สะดวกกว่าแต่มีค่าใช้จ่ายรายเดือน เปิดเพิ่มได้ทีหลังถ้าเปลี่ยนใจ

ไฟล์สำรองที่ก๊อปปี้ออกมา มีข้อมูลลูกค้าทั้งยิมและค่าลับของระบบอยู่ในไฟล์เดียว — เก็บไว้บนเครื่องที่มีรหัสผ่านเปิดเครื่อง อย่าวางไว้ในโฟลเดอร์ที่แชร์กัน และ อย่าส่งต่อทางแชตหรืออีเมล

งานประจำที่เจ้าของยิมต้องดู

ทุกสัปดาห์ — ก๊อปปี้ไฟล์สำรองล่าสุดออกนอกเครื่องหนึ่งครั้ง (ข้อ 3.5)

ทุกเดือน — สามอย่าง

1. ค่าต่ออายุ **VPS** และโดเมน — ถ้าขาด ระบบดับทั้งระบบ ไม่ใช่แค่ช้า
2. พื้นที่diskเหลือเท่าไร — รูปถ่ายสมาชิกและสลิปสะสมขึ้นเรื่อย ๆ
3. ชุดสำรองล่าสุดลงวันที่เมื่อไร — ถ้าค้างอยู่ที่หลายวันก่อน แปลว่างานสำรองล้มเหลวและต้องแจ้งทีม

ขั้นตอนของทุกข้ออยู่ในบทที่ 3 และสรุปรวมไว้ในภาคผนวก ก

ถ้าลูกค้าขอให้ลบข้อมูลของเขา

ทำได้ และมีขั้นตอนอยู่ในบท "ข้อมูลส่วนบุคคลของลูกค้า" ข้อสำคัญคือ การกด "ระงับสมาชิก" ในแอปไม่ใช่การลบ — เป็นการปิดไม่ให้เข้าใช้ แต่ข้อมูลยังอยู่ · การลบจริงต้องให้ทีมทำไบนนเครื่อง และชุดสำรองย้อนหลัง 7 วันจะยังมีข้อมูลนั้นอยู่จนครบรอบ

บทที่ 2 · ข้อมูลส่วนบุคคลของลูกค้า

บทนี้ไล่ทีละประเภทว่า เก็บอะไร · เก็บไว้ที่ไหน · ใครเปิดดูได้ · เก็บนานแค่ไหน · ลบอย่างไร ทุกบรรทัดตรงกับโค้ดของรุ่นที่จะติดตั้งรอบนี้ ไม่ได้เขียนจากความเข้าใจทั่วไป

ที่อยู่ของทุกอย่าง เครื่อง VPS ของยิมเครื่องเดียว · ฐานข้อมูล SQLite หนึ่งไฟล์ และโฟลเดอร์ไฟล์แนบ ทั้งคู่อยู่ใน volume ถาวรชื่อ `/data` ของคอนเทนเนอร์แอป · ไม่มีข้อมูลลูกค้าถูกส่งไปเก็บที่ผู้ให้บริการรายอื่น

2.1 ตารางสรุปตามประเภทข้อมูล

ข้อมูล	เก็บที่	ใครเปิดดูได้	อายุการเก็บ
ชื่อ-นามสกุล · เบอร์มือถือ · วันเกิด · ผู้ติดต่อฉุกเฉิน · รหัสสมาชิก · สถานะ	ฐานข้อมูล ตาราง members	พนักงานและเจ้าของยิม	เก็บถาวรจนกว่าจะลบด้วยมือ
อีเมลของสมาชิก (ไม่บังคับ)	ฐานข้อมูล ตาราง users ผูกกับสมาชิกผ่าน user_id	พนักงานและเจ้าของยิม	เก็บถาวร · ล้างช่องอีเมลในหน้าแก้ไข = ตัดบัญชีนั้นออกจากสมาชิกรายนี้
รูปถ่ายสมาชิก	ไฟล์ในดิสก์ /data/photos (ฐานข้อมูลเก็บแค่ชื่อไฟล์)	พนักงานและเจ้าของยิม (ผ่านหน้าจอที่ต้องล็อกอิน)	เก็บถาวร · ถ่ายใหม่ = เขียนทับไฟล์เดิม
บัตรสมาชิก (รูป PNG + QR)	ไม่ได้เก็บเป็นไฟล์ ระบบวาดขึ้นใหม่ทุกครั้งที่ขอ จากข้อมูลในฐานข้อมูล	คนที่ล็อกอินแล้วเท่านั้น · และผู้ถือลิงก์ "ส่งบัตรซ้ำ" ภายใน 7 วัน	— (ไฟล์ที่ลูกค้าเซฟไว้ในเครื่องตัวเองอยู่นอกความควบคุมของระบบ)
ประวัติเช็คอิน (เวลา จุดสแกน ผลลัพธ์)	ฐานข้อมูล ตาราง check_ins	พนักงานและเจ้าของยิม	เก็บถาวร
ประวัติการรับเงินและสิทธิ์ (แพ็คเกจ ยอด วิธีรับเงิน คนที่บันทึก)	ฐานข้อมูล ตาราง orders , entitlements	พนักงานและเจ้าของยิม	เก็บถาวร
รูปสลิปโอนเงิน	ไฟล์ในดิสก์ /data/slips	พนักงานและเจ้าของยิม	365 วัน แล้วลบอัตโนมัติ (cron วันที่ 1 ของเดือน 03:17)
เรื่องที่พนักงานแจ้ง + ภาพหน้าจอตอนแจ้ง (ภาพอาจติดชื่อและรูปของสมาชิก)	ฐานข้อมูล problem_reports + ไฟล์ /data/reports	เจ้าของยิมเท่านั้น · ทุกครั้งที่เปิดดูภาพถูกบันทึกไว้ว่าใครเปิด	180 วัน แล้วลบอัตโนมัติ (cron ทุกวัน 03:37)
บันทึกการเข้าเว็บของ Caddy (access.log) — มี IP ของผู้เข้าใช้ เวลา และหน้าที่เปิด	ไฟล์ใน volume ของ Caddy ไม่ได้อยู่ใน /data และไม่ได้อยู่ในชุดสำรอง	ผู้ดูแลเครื่องที่เข้า SSH ได้	หมุนไฟล์เองที่ 10 MiB เก็บ 5 ไฟล์ (ราว 50 MiB) แล้วทับของเก่า

2.1 (ต่อ) ตารางสรุปตามประเภทข้อมูล

ข้อมูล	เก็บที่	ใครเปิดดูได้	อายุการเก็บ
รหัสผ่านของพนักงาน เจ้าของยิม และสมาชิก	ฐานข้อมูล ตาราง <code>users</code> เก็บเป็น bcrypt ไม่ใช้รหัสจริง	ไม่มีใครอ่านกลับได้ รวมทั้งทีมพัฒนา	ถาวรจนกว่าจะเปลี่ยนรหัสใหม่
รหัสกล่องจดหมายของยิม / App password	ฐานข้อมูล ตาราง <code>mail_settings</code> เข้ารหัสด้วย AES-256-GCM คีย์อยู่ในไฟล์ <code>.env</code> (<code>SETTINGS_ENC_KEY</code>) แยกจากฐานข้อมูล	ไม่มีหน้าจอลงทะเบียน	จนกว่าจะแทนที่ด้วยรหัสใหม่
เซสชันการล็อกอิน	ตาราง <code>sessions</code> เก็บเฉพาะ ค่าแฮชของโทเค็น · คุณก็ <code>gym_session</code> เป็น <code>httpOnly + secure + sameSite strict</code>	—	12 ชั่วโมง แล้วหมดอายุเอง
ลิงก์ตั้งรหัสผ่าน / ยืนยันอีเมล / ลิงก์บัตร	ตาราง <code>password_setup_tokens</code> เก็บเป็นค่าแฮช (SHA-256)	ผู้ถือลิงก์	ลิงก์ที่เจ้าของยิมสร้าง 24 ชม. · ยืนยันอีเมล 24 ชม. · ลิงก์บัตรของสมาชิก 7 วัน · ทุกแบบ ใช้ได้ครั้งเดียว
บันทึกการกระทำ (audit) — ใครแก้ไขข้อมูลสมาชิก ใครเปิดดูภาพแจ้งปัญหา ใครออกบัตรใหม่ ใครเปลี่ยนสิทธิ์ ใครสร้างลิงก์เข้าช่วยเล่นให้สมาชิกคนไหน ใครแก้ไขเนื้อหาเครื่อง/โปรแกรม ใครอนุมัติข้อความความปลอดภัย และใครเข้า/ออกจากระบบเมื่อไหร่	ตาราง <code>audit_logs</code> เก็บค่าก่อนและหลังการแก้ไข · แถวเข้า/ออกจากระบบเก็บแค่สิทธิ์ของบัญชี ไม่มีหมายเลข IP และไม่มีชื่อเบราว์เซอร์	เจ้าของยิมอ่านเองได้ จากแท็บ "รายงาน" → "พนักงาน" · ผู้ดูแลเครื่องอ่านจากฐานข้อมูลได้เช่นกัน	เก็บถาวร
เนื้อหา "ช่วยเล่น" (เครื่อง โปรแกรม บทความ)	ตาราง <code>machines</code> , <code>programs</code> , <code>articles</code> , <code>safety_notices</code> + รูปเครื่องใน <code>/data/machines</code>	เปิดสาธารณะบางส่วน (หน้าเครื่องที่ผูกกับ QR ติดเครื่อง)	ถาวร — ไม่มีข้อมูลบุคคลอยู่ในนี้

2.2 ใครเข้าถึงอะไรได้ — สามชั้น

บทบาท	เข้าถึงได้	เข้าไม่ได้
พนักงาน (staff)	สมัครสมาชิก · บัตร · สแกนเช็คอิน · รับเงิน · แก้มข้อมูลสมาชิก · ดูประวัติของสมาชิกรายคน	ตั้งค่ายิม แพ็กเกจ บัญชีผู้ใช้ ค่าอีเมลของระบบ ภาพในเรื่องที่แจ้ง การระงับสมาชิก
เจ้าของยิม (admin)	ทุกอย่างของพนักงาน + ตั้งค่าทั้งหมด + ภาพในเรื่องที่แจ้ง	—
สมาชิก (member)	เฉพาะพอร์ทัล "ช่วยเล่น" ของตัวเอง · หมดอายุสมาชิก แล้วเข้าไม่ได้	ข้อมูลของสมาชิกคนอื่น หน้าจอของพนักงานทุกหน้า
ผู้ที่ไม่ได้ล็อกอิน	หน้าวิธีใช้เครื่องที่ผูกกับ QR ติดเครื่อง และหน้าบทความ	ทุกอย่างที่เหลือ

สองข้อที่ทำให้บทบาทเหล่านี้มีความหมายจริง

- ภาพหน้าจอในเรื่องที่พนักงานแจ้งอาจติดชื่อและรูปของสมาชิก ระบบจึงให้เฉพาะเจ้าของยิมเปิดดู และบันทึกทุกครั้งที่มีคนเปิดดู ว่าใครเปิดเมื่อไหร่ · แนวปฏิบัติ: เปิดเท่าที่จำเป็นต่อการแก้ปัญหา และลบเรื่องที่จบแล้วเป็นระยะ (ลบเรื่อง = ลบไฟล์ภาพด้วย)
- การแก้มข้อมูลสมาชิกทุกครั้งถูกบันทึก ว่าใครแก้มอะไรจากค่าอะไรเป็นค่าอะไร นี่คือเหตุผลที่เปิดให้พนักงานแก้มได้โดยไม่ต้องรอเจ้าของยิม

2.3 ข้อมูลที่ออกนอกเครื่อง

มีทางเดียว: อีเมลที่ระบบส่ง ผ่านกล่องจดหมาย Microsoft 365 ของยิมเอง (smtp.office365.com พอร์ต 587 เข้ารหัสด้วย STARTTLS) มีสี่ฉบับเท่านั้น

จดหมาย	ส่งถึง	มีข้อมูลบุคคลอะไรติดไปด้วย
บัตรสมาชิก	สมาชิกที่กรอกอีเมลไว้	ชื่อ รหัสสมาชิก วันหมดอายุ และ ไฟล์บัตรที่มีรูปถ่ายและ QR
ตั้งรหัสผ่านใหม่ (ลืมรหัสผ่าน)	พนักงาน/เจ้าของยิม	อีเมลของผู้รับ และลิงก์ 30 นาที
ยืนยันอีเมล / แจ้งผลอนุมัติบัญชี	ผู้ขอบัญชี	ชื่อและอีเมลของผู้ขอ
เมลทดสอบระบบ	กล่องของยิมเอง	ไม่มีข้อมูลลูกค้า

ระบบไม่เก็บสำเนาจดหมายที่ส่งออกไว้ในเครื่อง สำเนาที่มีคือสิ่งที่ Microsoft 365 ของยิมเก็บไว้ตามการตั้งค่าของกล่องนั้นเอง

2.4 การลบข้อมูล

สิ่งที่ป้อนในแอปทำจริง

- "ระงับสมาชิก" = ปิดไม่ให้เข้าใช้ ไม่ใช่การลบ — ชื่อ เบอร์ รูป และประวัติยังอยู่ครบ (เจตนา: ประวัติการรับเงินต้องตรวจย้อนได้)
- ล้างช่องอีเมลในหน้าแก้ไขข้อมูลสมาชิก = ตัดบัญชีเข้าใช้ "ช่วยเล่น" ออกจากสมาชิกรายนั้น และเตะออกจากระบบทันที
- "ลบเรื่องนี้" ในเรื่องที่แจ้ง = ลบทั้งเรื่องและไฟล์ภาพหน้าจอ ย้อนกลับไม่ได้
- ถ่ายรูปสมาชิกใหม่ = เขียนทับไฟล์รูปเดิม ไฟล์เก่าไม่เหลือ

ถ้าลูกค้าขอให้ลบข้อมูลทั้งหมดของเขา ต้องให้ผู้ดูแลเครื่องทำงานเครื่อง ตามลำดับนี้

1. บันทึกค่าขอไว้ (วันที่ ใครขอ ใครรับเรื่อง)
2. ลบไฟล์รูปถ่ายของสมาชิกรายนั้นใน `/data/photos` และสลิปที่เป็นของเขาใน `/data/slips`
3. ลบหรือลบชื่อออกจากแถวในฐานข้อมูล — ต้องตัดสินใจก่อนว่าจะเก็บประวัติการรับเงินแบบไม่ระบุตัวตนไว้หรือไม่ (เอกสารบัญชีของยิมอาจต้องใช้)
4. แจ้งว่า ชุดสำรองย้อนหลัง 7 วันยังมีข้อมูลนั้นอยู่ จนกว่าจะครบรอบและถูกทับไปเอง

การลบบัญชีกับบันทึกการกระทำ — เปลี่ยนไปในรุ่น `b8a9d96` ตาราง `audit_logs` มีคอลัมน์ `actor_id` ที่อ้างถึงตาราง `users` แบบบังคับ (foreign key) เดิมไม่มีกฎ "ลบตาม" เลย ทำให้ลบบัญชีที่เคยทำอะไรไว้ไม่ได้ · **migration 018 แก้เรื่องนี้แล้ว**

- แถวบันทึกจะอยู่รอดเมื่อบัญชีถูกลบ ระบบย้ายให้ไปชี้ที่ บัญชีตัวแทน `deleted-user` ซึ่งล็อกอินไม่ได้และไม่โผล่ในรายชื่อบัญชี · รายงานแสดงเป็นคำว่า "บัญชีที่ถูกลบ" — ประวัติว่าใครเตะข้อมูลของลูกค้าจึงไม่หายไปพร้อมบัญชี
- แต่ในทางปฏิบัติวันนี้ยังลบบัญชีไหนไม่ได้อยู่ดี เพราะตารางอื่นยังอ้าง `users` แบบไม่มีกฎ "ลบตาม" โดยเฉพาะ `password_setup_tokens` ซึ่งทุกบัญชีที่สร้างผ่านหน้า "บัญชีผู้ใช้" จะมีแถวอยู่เสมอ · migration 018 จึงเป็นเกราะที่เตรียมไว้ล่วงหน้า ไม่ใช่ของที่ทำงานอยู่แล้ว (ดูข้อ 4.12.3)
- วิธีที่ใช้อยู่และปลอดภัยกว่า คือ ระงับบัญชี ซึ่งปิดการเข้าใช้ทันทีและเก็บประวัติไว้ครบ

งานที่เหลือคือ migration 019 ให้ครอบตารางอื่น — อยู่ในรายการงานถัดไปแล้ว (ภาคผนวก ก.6)

ยังไม่มีปุ่ม "ลบสมาชิกถาวร" ในแอปโดยตั้งใจ เพราะการลบที่กดพลาดแล้วกู้ไม่ได้อยู่ห่างจากปุ่ม "ระงับ" เพียงหนึ่งบรรทัดในตาราง — ถ้าต้องการให้มี ให้สั่งทีมทำเป็นงานแยกพร้อมขั้นตอนยืนยันสองชั้น

2.4 (ต่อ) "เก็บ 180 วัน" แปลว่าอะไรจริง ๆ

ถ้าต้องตอบลูกค้าหรือหน่วยงานว่าข้อมูลอยู่ในระบบนานเท่าไร **ต้องนับสามชั้น ไม่ใช่ชั้นเดียว** — ตัวเลขในตารางข้อ 2.1 คือชั้นแรกเท่านั้น

ชั้น	อยู่ได้นานแค่ไหน
1 · ในระบบที่ใช้ทำงานอยู่	ตามตารางข้อ 2.1 — ภาพแจ้งปัญหา 180 วัน · สลิป 365 วัน · รูปถ่ายสมาชิกและประวัติ เก็บถาวรจนกว่าจะลบ
2 · ในชุดสำรองบนเครื่อง	ไม่เกิน 7 วัน หลังจากนั้น — ไฟล์สำรองรายวันเก็บย้อนหลัง 7 ชุด ของที่ถูกลบจากระบบวันนี้จึงยังอยู่ในชุดสำรองอีกไม่เกินหนึ่งสัปดาห์แล้วหายไปเอง
3 · ในสำเนาออกเครื่อง	ตามรอบที่ยิมก็อปปี้ออกไปและเก็บไว้เอง — ระบบตามไปลบให้ไม่ได้ · ถ้าลบข้อมูลของลูกค้ารายหนึ่งแล้ว ต้องตั้งใจลบสำเนาเก่าที่อยู่ในคอมพิวเตอร์ของยิมด้วย

คำตอบที่ถูกต้องเมื่อมีคนถาม คือ *"ในระบบเก็บ (ตามตาราง) · ในชุดสำรองอีกไม่เกิน 7 วัน · และในสำเนาที่ยิมเก็บไว้เองตามรอบที่ยิมกำหนด"* — ถ้าตอบแค่ตัวเลขชั้นแรก จะเป็นคำตอบที่ไม่ตรงกับความจริงของเครื่อง

ข้อแนะนำเชิงปฏิบัติ เก็บสำเนาออกเครื่องไว้ **ไม่เกิน 4 ชุดล่าสุด** แล้วลบของเก่าทิ้งเมื่อคัดลอกชุดใหม่ลงไป · จดไว้ว่าเก็บที่เครื่องไหน ใครเปิดได้ เพื่อให้ตอบได้จริงเมื่อถูกถาม

2.5 สามเรื่องที่ต้องรู้เกี่ยวกับความเสี่ยงของข้อมูลชุดนี้

1. ไฟล์สำรองหนึ่งไฟล์ = ข้อมูลลูกค้าทั้งยิม + ค่าลับของระบบ — สคริปต์สำรองคัดลอกทั้งโฟลเดอร์ `/data` และ สำเนาของ `.env` (ซึ่งมีทั้งกุญแจเซ็นบัตรและกุญแจถอดรหัสกล่องจดหมาย) ไว้ในไฟล์เดียวกัน · บนเครื่องไฟล์นี้เปิดอ่านได้เฉพาะ root · สำเนาที่ก๊อปไปต้องถือว่าเป็นของลับระดับสูงสุดของยิม เก็บบนเครื่องที่มีรหัสผ่าน ไม่ส่งต่อทางแชต · การมี `.env` อยู่ในไฟล์สำรองเป็นเรื่องจำเป็น ไม่ใช่ความสะดวก — ถ้าไม่มี กู้เครื่องแล้วบัตรสมาชิกทุกใบจะใช้ไม่ได้
2. ชุดสำรองเก็บบนเครื่องเดียวกับข้อมูลจริง จึงกันได้เฉพาะ "ลบผิด/ข้อมูลเสีย" ไม่ได้กันเครื่องหายทั้งเครื่อง — ทางที่เลือกไว้คือก๊อปไปออกนอกเครื่องสัปดาห์ละครั้ง (บทที่ 1 และข้อ 3.5)
3. บันทึกการเข้าเว็บของ **Caddy** ไม่ได้อยู่ในชุดสำรองและไม่ได้ถูกลบตามนโยบายข้อมูลลูกค้า — มันหมุนทับตัวเองตามขนาดไฟล์ ไม่ใช่ตามอายุ · ถ้าต้องการให้ครบตามนโยบาย ให้ตั้งอายุการเก็บของไฟล์นี้เพิ่มในบทที่ 3

บทที่ 3 · โครงสร้างพื้นฐาน

บทนี้เขียนโดย Infrastructure Engineer ของทีม เนื้อหาตรวจจากเครื่องจริง · ผู้อ่านที่ต้องลงมือคือคนที่เข้าเครื่องได้ · เจ้าของยิมอ่านเฉพาะข้อ 3.5 (สำรองข้อมูล) และ 3.9 (งานประจำ) ก็พอ

เอกสารนี้อธิบายเครื่อง ระบบปฏิบัติการ และการดูแลระบบของ **สุขฤทัย ฟิตเนส** สำหรับผู้ดูแลระบบ ข้อมูลทั้งหมดตรวจจากเครื่องจริงเมื่อ **16 กันยายน 2569 เวลา 21:21 น.** (รุ่นที่ติดตั้งอยู่ 44d5fa5)

ไม่มีรหัสผ่าน คีย์ หรือค่าลับใด ๆ ในเอกสารนี้ ทุกที่ที่ต้องอ้างถึงค่าลับจะเขียนเป็นชื่อตัวแปรเท่านั้น ถ้าพบว่ามีค่าจริงหลุดอยู่ในเอกสารฉบับใด ถือว่าเอกสารฉบับนั้นต้องถูกทำลายและออกใหม่

3.1 เครื่องและระบบปฏิบัติการ

รายการ	ค่า
ผู้ให้บริการ	Hostinger VPS แพคเกจ KVM 2
ชื่อเครื่อง	<code>srv1979069.hstgr.cloud</code>
IP (IPv4)	<code>187.127.107.148</code>
IP (IPv6)	<code>2a02:4780:5e:6cb3::1</code>
ระบบปฏิบัติการ	Ubuntu 26.04.1 LTS (เคอร์เนล <code>7.0.0-30-generic</code> , x86_64)
ซีพียู / แรม	2 vCPU / 7,935 MB
ดิสก์	96 GB — ใช้ไป 5.3 GB เหลือ 91 GB (6%)

ทุกอย่างของระบบอยู่บนเครื่องนี้เครื่องเดียว ไม่มีบริการภายนอกที่เก็บข้อมูลลูกค้า บริการภายนอกที่ระบบติดต่อออกไปมีเพียง **Let's Encrypt** (ขอใบรับรอง HTTPS) และ **GitHub** (ดาวน์โหลดโค้ดตอนอัปเดตรุ่น)

การเข้าเครื่อง ใช้ SSH ด้วยกุญแจเท่านั้น รหัสผ่านถูกปิดไว้ (PasswordAuthentication no) ลายนิ้วมือของเครื่อง (ใช้ตรวจสอบต่อถูกเครื่องจริง ไม่ใช่ความลับ):

```
ED25519  SHA256: IID3XSI+VJYuvyCWzJ8Zc91kLow4Vq76jCA38KWpXac
```

ถ้ากุญแจหาย: ตั้งรหัสผ่าน root ใหม่ผ่าน hPanel แล้ววาง public key ใหม่เข้า `/root/.ssh/authorized_keys`
การแนบ SSH key ผ่าน Hostinger API ใช้ไม่ได้กับเครื่องที่สร้างไปแล้ว (ทดสอบแล้ว ไม่มี action ใดเกิดขึ้น)
 ต้องทำผ่านตัวเครื่องเท่านั้น

3.2 Docker, Caddy และ HTTPS

ระบบรันเป็น 2 คอนเทนเนอร์ผ่าน Docker Compose ในโพลเดอร์ `/srv/gym`

เวอร์ชัน / อิมเมจ		หน้าที่
Docker Engine	29.8.0	
Docker Compose	v5.5.1	
คอนเทนเนอร์ <code>app</code>	สร้างเองจาก <code>Dockerfile</code> ฐาน <code>node:24-bookworm-slim</code> (Node 24.21.0)	ตัวระบบทั้งหมด ฟังพอร์ต 3000 ภายในเครือข่ายของ Docker เท่านั้น
คอนเทนเนอร์ <code>caddy</code>	<code>caddy:2-alpine</code> (Caddy v2.11.4)	รับ 80/443 จากอินเทอร์เน็ต ต่อ HTTPS และส่งต่อให้ <code>app</code>

`app` **ไม่ได้เปิดพอร์ตออกอินเทอร์เน็ตเลย** — เข้าถึงได้ทางเดียวคือผ่าน Caddy นี่คือขอบเขตความปลอดภัยขั้นแรกของระบบ

ไฟล์ตั้งค่าของ Caddy อยู่ที่ `deploy/Caddyfile` เนื้อหาโดยสรุป:

```
{ $APP_DOMAIN } {
  encode gzip
  reverse_proxy app:3000 {
    header_up X-Forwarded-For {remote_host}
    header_up X-Forwarded-Proto {scheme}
  }
  log { output file /data/access.log { roll_size 10MiB roll_keep 5 } }
}
```

HTTPS อัตโนมัติ Caddy ขอใบรับรองจาก Let's Encrypt เองและต่ออายุเองก่อนหมดอายุ **ไม่ต้องมีใครจำวันหมดอายุ** ในปัจจุบัน: `CN=srv1979069.hstgr.cloud` ออกโดย Let's Encrypt ใช้ได้ถึง **13 ธันวาคม 2569** เงื่อนไขที่ทำให้ต่ออายุสำเร็จคือ DNS ของ `srv1979069.hstgr.cloud` ต้องชี้มาที่เครื่องนี้ และพอร์ต 80/443 ต้องเปิดอยู่ — ถ้าวันหนึ่งใบรับรองหมดอายุ ให้ตรวจสอบข้อนี้ก่อนเสมอ

`X-Forwarded-For` ที่ Caddy ใส่ให้ คู่กับ `TRUST_PROXY=1` ผังแอป คือสิ่งที่ทำให้ระบบรู้ IP จริงของผู้ใช้ ถ้าตั้งผิด ผู้ใช้ทุกคนจะถูกนับรวมเป็นคนเดียวในระบบจำกัดจำนวนครั้ง แล้วล็อกทั้งยิมออกจากระบบพร้อมกัน

3.3 โฟลเดอร์และที่เก็บข้อมูลถาวร

ที่อยู่	คืออะไร	หายได้ไหมถ้าสร้างเครื่องใหม่
<code>/srv/gym</code>	โค้ดที่ clone จาก GitHub + ไฟล์ <code>.env</code>	โค้ดดึงใหม่ได้ · <code>.env</code> หายไม่ได้
<code>/srv/gym/.env</code>	ค่าตั้งค่าและค่าลับ สิทธิ์ 600 เจ้าของ <code>root</code>	ห้ามหาย (ดูข้อ 3.4)
Docker volume <code>gym_gym-data</code>	ข้อมูลลูกค้าทั้งหมด	ห้ามหาย
Docker volume <code>gym_caddy-data</code>	ใบรับรอง HTTPS + <code>access.log</code>	สร้างใหม่ได้ (ขอใบรับรองใหม่)
Docker volume <code>gym_caddy-config</code>	ค่าภายในของ Caddy	สร้างใหม่ได้
<code>/var/backups/gym</code>	ไฟล์สำรองข้อมูล	ควรคัดลอกออกนอกเครื่อง
<code>/root/RUNBOOK.md</code>	คู่มือปฏิบัติงานฉบับย่อของผู้ดูแล	

ภายในคอนเทนเนอร์ `app` volume `gym_gym-data` ถูก mount ไว้ที่ `/data` และมีของอยู่ดังนี้:

ที่อยู่	เก็บอะไร
<code>/data/gym.sqlite (+ -wal, -shm)</code>	ฐานข้อมูลทั้งหมด
<code>/data/photos</code>	รูปถ่ายสมาชิก
<code>/data/slips</code>	สลิปโอนเงิน
<code>/data/logo</code>	โลโก้ยิม
<code>/data/reports</code>	ภาพหน้าจอที่แนบมากับใบแจ้งปัญหา
<code>/data/machines</code>	รูปเครื่องออกกำลังกายในพอร์ทัลสมาชิก (ไม่ใช่ข้อมูลบุคคล)

บนตัวเครื่อง volume นี้อยู่ที่ `/var/lib/docker/volumes/gym_gym-data/_data` ขนาดปัจจุบันประมาณ 2.6 MB

จุดที่สับสนได้ง่ายและต้องระวัง คอนเทนเนอร์ `caddy` ก็มีโฟลเดอร์ชื่อ `/data` ของตัวเองเหมือนกัน แต่เป็นคนละ volume `access.log` ของ Caddy (ปัจจุบัน ~3.4 MB) อยู่ใน `gym_caddy-data` จึงไม่ได้อยู่ในชุดสำรองข้อมูล และหมุนไฟล์เองที่ 10 MiB เก็บ 5 ไฟล์ (สูงสุดราว 50 MiB) ไฟล์นี้บันทึก IP ของผู้ใช้เว็บ ถ้าจะทำเรื่องข้อมูลส่วนบุคคลให้ครบ ต้องนับไฟล์นี้ด้วย

กฎเหล็กของการเขียนไฟล์: ทุกพาธที่ระบบเขียนไฟล์ลงไปต้องอยู่ใต้ `/data` เพราะทุกอย่างนอก `/data` จะหายไปตอน build ใหม่ โดยไม่มี error ที่ไหนเลย เคยเกิดขึ้นจริงกับโลโก้มาแล้ว ปัจจุบันมีเทสต์ `tests/deploy-paths.test.js` คอยจับถ้ามีพาธใหม่ตกหล่น

3.4 ไฟล์ .env — มีคีย์อะไร และอะไรห้ามเปลี่ยน

ไฟล์อยู่ที่ `/srv/gym/.env` สิทธิ์ `600` เจ้าของ `root` อ่านได้เฉพาะผู้ที่เข้าเครื่องได้ ตารางนี้แสดงเฉพาะคีย์ที่ไม่มีค่าใด ๆ

คีย์	คืออะไร	เปลี่ยนได้ไหม
<code>CARD_SIGNING_SECRET</code>	กุญแจที่ใช้เซ็น QR บนบัตรสมาชิก	🔴 ห้ามเปลี่ยนเด็ดขาดหลังแจกบัตรใบแรก
<code>SETTINGS_ENC_KEY</code>	กุญแจถอดรหัสรหัสกล่องจดหมายที่เจ้าของยิมกรอกในแท็บ "อีเมลของระบบ"	🔴 ห้ามเปลี่ยน เปลี่ยนแล้วรหัสที่เก็บไว้ถอดไม่ออก ต้องกรอกใหม่ · ต้องยกไปด้วยเมื่อย้ายเครื่อง
<code>PILOT_MODE</code>	โหมดทดลอง ยังไม่รับชำระเงินผ่าน PromptPay	🔴 อย่าเคลียร์เองตรง ๆ (ดูด้านล่าง)
<code>APP_DOMAIN</code> , <code>APP_ORIGIN</code>	ชื่อโดเมนของระบบ ใช้ทั้งขอใบรับรองและตรวจที่มาของคำขอ	🟡 เปลี่ยนพร้อมกันทั้งคู่เมื่อย้ายโดเมนเท่านั้น
<code>TRUST_PROXY</code>	จำนวนพรีอ็อกซีที่อยู่หน้าแอป (ต้องเป็น 1 เพราะมี Caddy ชั้นเดียว)	🟡 อย่าแตะถ้าไม่ได้เปลี่ยนโครงสร้าง
<code>PROMTPAY_ID</code>	บัญชีรับเงิน PromptPay	🟢 เจ้าของยิมกำหนด
<code>NODE_ENV</code> , <code>PORT</code> , <code>HOST</code>	ค่ามาตรฐานของการรันแบบ production	🟡 ไม่ต้องแตะ
<code>DATABASE_PATH</code> , <code>SLIP_STORAGE_PATH</code> , <code>SLIP_RETENTION_DAYS</code>	ที่เก็บฐานข้อมูล/สลิป และอายุการเก็บสลิป (365 วัน)	⚠️ ดูหมายเหตุ
<code>ADMIN_EMAIL</code>	ใช้ครั้งเดียวตอนติดตั้งเพื่อสร้างแอดมินคนแรก แล้วถูกล้างค่า	🟢 วางอยู่แล้ว ปกติ
<code>ALLOW_DESTRUCTIVE_ROLLBACK</code>	สวิตช์กันการถอย migration โดยไม่ตั้งใจ	🔴 ปลอยวางไว้
<code>SMTP_HOST</code> ; <code>SMTP_PORT</code> ; <code>SMTP_SECURE</code> ; <code>SMTP_USER</code> ; <code>SMTP_PASSWORD</code> ; <code>MAIL_FROM</code>	คีย์ตกค้างจากรุ่นเก่าที่เคยส่ง OTP ทางอีเมล	✅ ลบออกจากเครื่องแล้ว เมื่อ 17 ก.ย. 2569 (คีย์เหลือ 15 ตัว)

3.4 (ต่อ) คีย์ที่ต้องระวังเป็นพิเศษ

เรื่อง `CARD_SIGNING_SECRET` ลายเซ็นบน QR ของบัตรสมาชิกทุกใบผูกกับค่านี ถ้าเปลี่ยนเมื่อไร บัตรที่แจกไปแล้วทุกใบจะสแกนไม่ผ่านพร้อมกัน เจียบ ๆ ไม่มีข้อความบอกสาเหตุ ทางกู้คืนมีทางเดียวคือเอาค่าเดิมกลับมา หรือออกบัตรใหม่ให้สมาชิกทุกคน — นี่คือเหตุผลที่ขั้นตอนกู้คืนข้อมูลต้องคืน `.env` ก่อนเสมอ

เรื่อง `PILOT_MODE` ถ้าเคลียร์ค่านีทั้งขณะที่ยังไม่มี `PROMPTPAY_ID` แอปจะไม่ยอมมุดและเว็บล่มทันที (เป็นพฤติกรรมที่ตั้งใจ กันเปิดรับเงินโดยไม่มีบัญชีรับเงิน) เส้นทางเปิดใช้จริงที่ถูกต้องคือรัน `deploy/bootstrap.sh` โดยไม่ใส่ `--pilot` ซึ่งจะถาม PromptPay ID แล้วตั้งค่าให้เองอย่างถูกลำดับ

หมายเหตุสำคัญเรื่องพาร `docker-compose.yml` กำหนด `DATABASE_PATH`, `SLIP_STORAGE_PATH`, `PHOTO_STORAGE_PATH`, `LOGO_STORAGE_PATH`, `REPORT_STORAGE_PATH` ไว้ในส่วน `environment`: ซึ่งหับค่าจาก `.env` เสมอ การแก้พารเหล่านี้ใน `.env` จึงไม่มีผลใด ๆ ถ้าต้องเปลี่ยนจริงต้องแก้ที่ `docker-compose.yml`

สองกับดักของ `.env` ที่เจอมาแล้วบนเครื่องจริง

1. แก้ `.env` ก่อนรัน `up.sh` เสมอ ไม่ใช่หลัง — คอนเทนเนอร์อ่านไฟล์นี้ตอนสร้างเท่านั้น ตั้งคีย์ใหม่หลัง `up.sh` แล้วแอปจะยังไม่เห็นค่านั้นจนกว่าจะสร้างคอนเทนเนอร์ใหม่อีกรอบ
2. ห้ามรัน `docker compose exec app npm run env:set` — `env-set.js` หาไฟล์ `.env` เทียบกับโพลเดอร์ปัจจุบัน ซึ่งในคอนเทนเนอร์คือ `/app` ไม่ใช่ `/srv/gym` ผลคือเขียนไฟล์ใหม่ลงใน layer ของอิมเมจ ซึ่งหายไปด้วย `rebuild` โดยไม่มี `error` ให้รันบนตัวเครื่องที่ `/srv/gym` ในฐานะ `root` หรือในคอนเทนเนอร์ชั่วคราวที่ `mount /srv/gym` เข้ามาแล้วชี้ `ENV_FILE` มาที่นั่น

วิธีแก้ค่าใน `.env` อย่างปลอดภัย ใช้สคริปต์ของระบบ ไม่ต้องเปิดไฟล์แก้มือ — มันเขียนทับเฉพาะบรรทัดที่ระบุเขียนแบบ `atomic` และไม่พิมพ์ค่ากลับออกมา:

```
cd /srv/gym
npm run env:set -- KEY=VALUE
npm run env:set -- --stdin KEY      # พิมพ์ค่าแล้ว Enter ค่าจะไม่ปรากฏในบรรทัดคำสั่งและไม่ลง history
npm run env:set -- --list           # ดูว่ามีคีย์อะไรบ้าง โดยไม่แสดงค่า
```

แท็บ "อีเมลของระบบ" อยู่ในรุ่น `b8a9d96` แล้ว เจ้าของยิมกรอกรหัสกล่องจดหมายเองผ่านหน้าเว็บ ระบบเก็บเข้ารหัสไว้ในฐานข้อมูลด้วยคีย์ `SETTINGS_ENC_KEY` ในตารางด้านบน — ค่ารหัสไม่เคยอยู่ใน `.env` และไม่มีหน้าจอไหนแสดงกลับ

3.5 การสำรองข้อมูล การกู้คืน และงานตามเวลา

งานตามเวลา (`/etc/cron.d/gym-pilot`)

เวลา (เวลาเครื่อง = UTC)	ทำอะไร	บันทึกผลที่
ทุกวัน 02:17	สำรองข้อมูล <code>deploy/backup.sh</code>	<code>/var/log/gym-backup.log</code>
ทุกวัน 03:37	ลบภาพใบแจ้งปัญหาที่เกิน 180 วัน	<code>/var/log/gym-prune.log</code>
วันที่ 1 ของเดือน 03:17	ลบสลิปที่เกิน 365 วัน	<code>/var/log/gym-prune.log</code>

งานลบทั้งสองใช้ `flock` ตัวเดียวกัน (`/run/gym-maintenance.lock`) จึงไม่มีทางทำงานทับกัน

ชุดสำรองข้อมูล

ไฟล์อยู่ที่ `/var/backups/gym/gym-<วันเวลา>-<pid>.tar.gz` เก็บ 7 วัน (ไฟล์เก่านั้นถูกลบอัตโนมัติ) ปัจจุบันมี 15 ไฟล์ สคริปต์คัดลอก `/data` ทั้งหมดด้วย `docker compose cp app:/data/.` จึงเก็บทุกอย่างได้ `/data` โดยอัตโนมัติ ไม่ได้ไล่ชื่อโฟลเดอร์ทีละอัน แต่ละไฟล์มี 3 ส่วน:

ในไฟล์สำรอง	คือ
<code>./data/</code>	ฐานข้อมูล + WAL + <code>photos</code> + <code>slips</code> + <code>logo</code> + <code>reports</code>
<code>./environment</code>	สำเนาของ <code>.env</code> (รวม <code>CARD_SIGNING_SECRET</code>)
<code>./revision</code>	commit hash ของโค้ดที่ใช้อยู่ตอนถ่ายไฟล์นั้น

ระหว่างสำรอง แอปถูกหยุดประมาณ 1 วินาที เพื่อให้ฐานข้อมูลนิ่ง (ช่วงนั้นเว็บอาจตอบ 502 ชั่วขณะ) แล้วเปิดกลับเสมอแม้การคัดลอกล้มเหลว · ก่อนรับไฟล์ สคริปต์ตรวจ `PRAGMA integrity_check` ของฐานข้อมูลที่คัดลอกมา ไฟล์ที่ผ่านเท่านั้นจึงถูกเก็บ

ไฟล์สำรองมีทั้งข้อมูลลูกค้าและค่าลับของระบบ สิทธิ์ไฟล์เป็น 600 เจ้าของ root ถ้าคัดลอกออกนอกเครื่อง (ซึ่งควรทำ) ต้องเก็บในที่ที่ปลอดภัยเทียบเท่ากัน

ข้อจำกัดที่ต้องรู้ และการก๊อปปี้ออกนอกเครื่อง

ชุดสำรองอยู่บนดิสก์ก่อนเดียวกับระบบ ป้องกันได้เฉพาะกรณี "ลบผิด/ข้อมูลเสีย" ไม่ได้ป้องกันกรณีเครื่องหาย ดิสก์พัง หรือบัญชีถูกปิด เจ้าของยืมตัดสินใจแล้วว่า จะก๊อปปี้ออกนอกเครื่อง สัปดาห์ละหนึ่งครั้ง ลงคอมพิวเตอร์ของยืม (ไม่มีค่าใช้จ่ายเพิ่ม) ทำจากเครื่องที่มีกุญแจ SSH ด้วยคำสั่งเดียว:

```
scp -i <กุญแจ> root@187.127.107.148:/var/backups/gym/gym-*.tar.gz ./
```

ไฟล์ที่ได้ มีข้อมูลลูกค้าทั้งยืมและสำเนา `.env` เก็บไว้บนเครื่องที่มีรหัสผ่านเปิดเครื่อง อย่าวางในโฟลเดอร์ที่แชร์กัน และ อย่าส่งต่อทางแชตหรืออีเมล · ทางเลือกที่สะดวกกว่าแต่มีค่าใช้จ่ายรายเดือนคือเปิด snapshot/backup ของ Hostinger เพิ่มอีกชั้น

ขั้นตอนกู้คืน

```
cd /srv/gym
mkdir -p /tmp/restore && tar xzf /var/backups/gym/gym-<วันเวลา>.tar.gz -C /tmp/restore
git fetch origin release/pilot
git checkout -f --detach "$(cat /tmp/restore/revision)"      # โค้ดรุ่นเดียวกับข้อมูล
install -m 600 /tmp/restore/environment .env                # คัด .env ก่อนเสมอ
docker compose up -d --no-start app
docker compose cp /tmp/restore/data/. app:/data/
./deploy/up.sh --wait --wait-timeout 180
rm -rf /tmp/restore
```

ลำดับสำคัญ: คัด `.env` ก่อนเปิดระบบเสมอ ถ้า `CARD_SIGNING_SECRET` ไม่ใช่ค่าเดิม บัตรสมาชิกที่แจกไปแล้วทุกใบ จะใช้ไม่ได้ และต้อง checkout โค้ดตาม `./revision` ด้วย เพราะข้อมูลที่กู้มาอาจมีโครงสร้างตรงกับโค้ดรุ่นนั้นเท่านั้น

3.6 พอร์ตและไฟร์วอลล์

ไฟร์วอลล์คือ `ufw` สถานะ **active** นโยบาย ปฏิเสธขาเข้าทั้งหมดโดยปริยาย อนุญาตขาออกทั้งหมด

พอร์ต	ใช้ทำอะไร
22/tcp	SSH (กุญแจเท่านั้น)
80/tcp	HTTP — ใช้เปลี่ยนเส้นทางไป HTTPS และให้ Let's Encrypt ตรวจสอบสิทธิ์โดเมน ห้ามปิด ไม่จั่นใบรับรองต่ออายุไม่ได้
443/tcp	HTTPS

เปิดครบทั้ง IPv4 และ IPv6 · ไม่มีพอร์ตอื่นเปิดออกอินเทอร์เน็ต · ฐานข้อมูลไม่ได้เปิดพอร์ตใด ๆ เพราะเป็น SQLite ที่เป็นไฟล์ในคอนเทนเนอร์

ข้อควรระวัง พอร์ตที่ Docker เปิดเอง (ports: ใน compose) ห้าม `ufw` ได้ ปัจจุบันโปรเจกต์นี้เปิดแค่ 80/443 ของ Caddy ซึ่งตั้งใจให้เปิดอยู่แล้ว อย่างคอนเทนเนอร์อื่นที่มี ports: มารันบนเครื่องนี้ เพราะจะเปิดพอร์ตออกอินเทอร์เน็ตโดยที่ `ufw status` ไม่แสดงให้เห็น

การเชื่อมต่อขาออกที่ระบบต้องใช้: 443 (Let's Encrypt, GitHub) และ 587 (ส่งอีเมลผ่าน Office 365 เมื่อเปิดใช้) — ทดสอบแล้วว่าต่อ `smtp.office365.com:587` พร้อม STARTTLS ได้จากเครื่องนี้

3.7 ขั้นตอนอัปเดตรุ่น

```
cd /srv/gym
bash deploy/backup.sh # สำรองก่อนเสมอ (แอปหยุดราว 1 วินาที)
git fetch origin release/pilot
git checkout -f --detach <COMMIT SHA เดิม>
./deploy/up.sh --wait --wait-timeout 180
```

ต้องใช้ `./deploy/up.sh` **ไม่ใช่** `docker compose up` **ตรง ๆ** เพราะสคริปต์อ่านเลข commit จาก git แล้วส่งเข้าไปฝังในหน้าเว็บเป็น `APP_REVISION` ใ้แก้ปัญหากที่พนักงานส่งจึงติดเลขรุ่นมาด้วย ถ้า build ด้วย `compose` ตรง ๆ ช่องนั้นจะว่าง และรายงานที่มาถึงหลังอัปเดตรุ่นถัดไปจะไม่มีใครรู้ว่าพูดถึงหน้าจอรุ่นไหน

อย่าสั่ง `chmod +x deploy/up.sh` ถ้าเจอ `Permission denied` ให้ใช้ `sh deploy/up.sh` แทน — การ `chmod` เป็นการแก้ mode ของไฟล์ที่ git ติดตามอยู่ สคริปต์จะมองว่า `working tree` สกปรกแล้วประทับเลขรุ่นเป็น `<sha>-dirty` ทั้งที่โค้ดตรงกับ commit ทุกบรรทัด (ตั้งแต่รุ่น `ecb2a5d` สิทธิ์ไฟล์ถูกแก้ใน repo แล้ว จึงไม่ต้อง `chmod` อีก)

ตรวจหลังอัปเดตทุกครั้ง

1. บรรทัด `{"event": "deploy_build", "app_revision": "..."}` ต้องไม่ลงท้าย `-dirty`
2. `curl -s https://srv1979069.hstgr.cloud/api/health` ต้องได้ `{"status": "ok"}`
3. `docker compose ps` ต้อง healthy ทั้งสองคอนเทนเนอร์
4. เลข migration ตรงกับที่รุ่นนั้นระบุ และ `pragma foreign_key_check` ต้องว่าง
5. บัญชีผู้ดูแลยังอยู่ครบ
6. ถ้ามอบนั้นแก้หน้าตา ให้ดึง stylesheet ที่เสิร์ฟจริงมาตรวจ ไม่ใช่ดูแค่ในโค้ด

ย้อนรุ่น `git checkout -f --detach <SHA เดิม>` แล้ว `./deploy/up.sh` — ฐานข้อมูลด้วยเฉพาะเมื่อรุ่นนั้นมี migration

ถ้า **build** ล้มเหลว ให้ดู `.dockerignore` เป็นที่แรก เคยเกิดสองครั้งและทั้งสองครั้งไม่ใช่ความผิดของ `Dockerfile` : ครั้งแรกโฟลเดอร์ `shared/` ไม่ได้ถูก `COPY` ครั้งที่สอง `.dockerignore` กัน `docs` ไว้จนคู่มือ PDF ไม่เข้า build context · ไม่มีเทสไหนจับได้เพราะไม่มีชุดไหนรัน `docker build` ปัจจุบันมีเทสต์ `tests/dockerfile-copies.test.js` คอยจับแล้ว

3.8 ขั้นตอนย้ายไปเครื่องใหม่

ใช้เมื่อเปลี่ยนผู้ให้บริการ อัปเกรดเครื่อง หรือกู้จากเครื่องที่พังทั้งเครื่อง

ก่อนเริ่ม เตรียม 3 อย่าง: ไฟล์สำรองล่าสุด (`gym-*.tar.gz`), ลิสท์แก้ DNS ของโดเมน, และ SSH public key ที่จะใช้กับเครื่องใหม่

1. **เครื่องใหม่** สั่ง VPS สเปคเท่าเดิมหรือสูงกว่า ระบบปฏิบัติการ **Ubuntu 22.04 / 24.04 / 26.04** (ตัวติดตั้งรับเฉพาะสามรุ่นนี้) และต้องเป็นเครื่องเปล่าที่ไม่มีบริการอื่นรันอยู่
 2. **ติดตั้ง** รันคำสั่งติดตั้งชุดเดียวกับที่ใช้ตอนแรก โดย pin commit เดียวกับที่อยู่ในไฟล์ `./revision` ของชุดสำรอง — ตัวติดตั้งจะลง Docker, ตั้ง ufw, ตั้ง cron และสร้าง `.env` ใหม่ให้
 3. **คืนข้อมูล** ทำตามขั้นตอนกู้คืนในข้อ 3.5 — ใช้ `.env` จากไฟล์สำรอง ไม่ใช่ `.env` ที่ตัวติดตั้งเพิ่งสร้าง เพราะจะได้ `CARD_SIGNING_SECRET` ตัวเดิมกลับมา ไม่อย่างนั้นบัตรสมาชิกที่แจกไปแล้วใช้ไม่ได้ทั้งหมด
 4. **ย้ายโดเมน** ชี้ DNS ของ `srv1979069.hstgr.cloud` (หรือโดเมนใหม่) มาที่ IP ใหม่ ถ้าเปลี่ยนชื่อโดเมนต้องแก้ `APP_DOMAIN` และ `APP_ORIGIN` ให้ตรงกันทั้งคู่ · รอ Caddy ขอใบรับรองใหม่ (ไม่กี่นาที) แล้วตรวจ `https://<โดเมน>/api/health`
 5. **ปิดเครื่องเก่า** ต่อเมื่อเครื่องใหม่ผ่านครบ: health 200, ล็อกอินได้, สแกนบัตรใบเดิมของสมาชิกผ่าน, รูปสมาชิกและโลโก้ขึ้นครบ · การสแกนบัตรใบเดิมได้คือข้อพิสูจน์ว่า `CARD_SIGNING_SECRET` ถูกคืนถูกต้อง ควรทดสอบก่อนปิดเครื่องเก่าเสมอ
-

3.9 สิ่งที่ต้องดูเป็นประจำ

ความถี่	ตรวจสอบอะไร	เกณฑ์ / วิธีทำ
ทุกเดือน	พื้นที่ดิสก์	<code>df -h /</code> — ปัจจุบันใช้ 6% ถ้าเกิน 70% ให้ดูอิมเมจเก่าด้วย <code>docker image prune -a</code> และขนาด <code>/var/backups/gym</code>
ทุกสัปดาห์	ไฟล์สำรองออกนอกเครื่อง	คัดลอกไฟล์ล่าสุดลงคอมพิวเตอร์ของยิมด้วย <code>scp</code> (ดูข้อ 3.5) — ข้อนี้เจ้าของยิมทำเองได้
ทุกเดือน	ผลของงานอัตโนมัติ	<code>tail /var/log/gym-backup.log /var/log/gym-prune.log</code> ต้องไม่มี error
ทุกเดือน	ใบรับรอง HTTPS	ปกติ Caddy ต่ออายุเอง ตรวจสอบว่าวันหมดอายุขยับไปข้างหน้าทุกไตรมาส
เมื่อมีแจ้ง	รีบูตเคอร์เนล	<code>ls /var/run/reboot-required</code> — ตอนนี้เครื่องค้างสถานะนี้อยู่ นัดเวลากับเจ้าของยิม แล้ว <code>reboot</code> ใช้เวลาไม่ถึงนาที ระบบขึ้นเองเพราะคอนเทนเนอร์ตั้ง <code>restart: unless-stopped</code>
ทุกเดือน	แพตช์ระบบปฏิบัติการ	<code>apt-get update && apt-get -y upgrade</code> แล้วดูว่ามี <code>reboot-required</code> ตามมาไหม
ก่อนหมดอายุ	ต่ออายุ VPS และโดเมน	ดูวันหมดอายุในบัญชี Hostinger · ถ้า VPS หมดอายุ เครื่องถูกปิดและข้อมูลอาจถูกลบ ควรเปิดต่ออายุอัตโนมัติไว้
ทุกไตรมาส	ซ้อมกู้คืน	กู้ไฟล์สำรองลงเครื่องทดสอบหนึ่งครั้ง — ไฟล์สำรองที่ไม่เคยกู้ คือไฟล์สำรองที่ยังไม่รู้ว่าจะใช้ได้ไหม

3.10 คำสั่งที่ใช้บ่อย

```
# สถานะระบบ
cd /srv/gym && docker compose ps
curl -s https://srv1979069.hstgr.cloud/api/health

# ดู log ของแอป 100 บรรทัดล่าสุด
docker compose logs --tail 100 app

# รุ่นที่ติดตั้งอยู่ตอนนี้
git rev-parse HEAD

# สำรองข้อมูลทันที (แอปหยุดราว 1 วินาที)
bash deploy/backup.sh

# รีสตาร์ทแอปอย่างเดียว
docker compose restart app

# ออกลิงก์ตั้งรหัสผ่านให้บัญชีผู้ดูแล (ใช้ครั้งเดียว อายุ 24 ชม.)
docker compose exec app npm --silent run admin:set-password-link -- <อีเมล>
```

3.11 บัญชีและรุ่นที่ติดตั้งอยู่

ค่า ณ 17 กันยายน 2569	
รุ่นที่ติดตั้ง (APP_REVISION)	856846c (ขึ้นเครื่อง 17 ก.ย. 2569 · ก่อนหน้านี้คือ 115fb17) · schema_migrations ถึง 17
บัญชีในระบบ	akerit@suklutai.co.th (เจ้าของยิม) · akerit@kencorp.net (เจ้าของยิม) · info@suklutai.co.th (พนักงาน)
คีย์ใน .env	15 ตัว (หลังลบ SMTP_* และ MAIL_FROM 6 ตัว) · SETTINGS_ENC_KEY ตั้งแล้ว
กล่องจดหมายของยิม	ยังไม่ได้กรอกรหัส ในแท็บ "อีเมลของระบบ" — จนกว่าจะกรอก ลิงก์ลิ้มรสผ่านและบัตรทางอีเมล จะยังไม่ออก
เนื้อหาพอร์ทัล	เครื่อง 6 · โปรแกรม 1 · บทความ 3 · ข้อความความปลอดภัย นำเข้าแล้ว

ตัวเลขในตารางนี้เป็นค่า ณ วันที่เขียน ตรวจสอบจริงได้ด้วย `git rev-parse HEAD` บนเครื่อง และหน้า "ผู้ใช้และสิทธิ์" ในแอปสำหรับรายชื่อบัญชี

บทที่ 4 · แอปพลิเคชัน

บทนี้เขียนโดย Full Stack Developer ของทีม · ผู้อ่านคือคนที่ต้องแก้โค้ดหรือตรวจสอบระบบ เจ้าของยิมไม่จำเป็นต้องอ่านบทนี้ ยกเว้นข้อ 4.6 ถ้าอยากรู้ว่าข้อมูลถูกปกป้องอย่างไร

เอกสารนี้ไม่มีรหัสผ่าน คีย์ หรือค่าลับใด ๆ และไม่ควรมี · ที่เก็บค่าลับคือ `.env` บนเครื่องเท่านั้น

4.1 สแตกและเวอร์ชัน

ส่วน	ใช้อะไร	หมายเหตุ
ภาษา/รันไทม์	Node.js 24 (engines: >=24.15.0 <25)	ใช้ ES modules ("type": "module") ทั้งโปรเจกต์
เว็บเซิร์ฟเวอร์	Express 5	server/app.js เป็นที่ประกอบทุก route
ฐานข้อมูล	SQLite ผ่าน node:sqlite ที่มาในตัว Node 24	ไม่มี dependency ฐานข้อมูล ไม่ต้องคอมไพล์ ไม่มี native module ให้พึ่งตอนอัปเดต
หน้าเว็บ	React 19 + Vite (build เป็นไฟล์สแตติกใน dist/)	ไม่มี server-side React — ยกเว้นหน้าเครื่องที่เซิร์ฟเวอร์เรนเดอร์เป็น HTML ธรรมดา
ความปลอดภัย HTTP	helmet 8	CSP, HSTS (เฉพาะ production)
ตรวจสอบข้อมูลเข้า	zod 4	ทุก request body ผ่าน schema ก่อนแตะฐานข้อมูล
รหัสผ่าน	bcryptjs 3	เป็น JS ล้วนโดยตั้งใจ — ดูข้อ 4.6
อีเมล	nodemailer 10	SMTP ไปที่ Office 365 ของยิม
รูปบัตร	@napi-rs/canvas + qrcode	วาดบัตร PNG 1080×1350 ฝั่งเซิร์ฟเวอร์
อ่าน QR ใน เบราร์เซอร์	BarcodeDetector ของเบราร์เซอร์ ถ้าไม่มี ใช้ jsQR	jsQR แยกเป็นก้อนต่างหาก โหลดตอนเปิดกล่องเท่านั้น
ฟอนต์	@fontsource/noto-sans-thai , @fontsource/ibm-plex-sans	ฝังมากับแอป ไม่ดึงจาก Google Fonts ตอนใช้งาน

ไม่มีบริการภายนอกที่ระบบต้องพึ่งตอนทำงาน ยกเว้นสองอย่าง: กล่องอีเมล Office 365 ของยิม (สำหรับส่งจดหมาย) และ YouTube (เฉพาะตอนสมาชิกกดเล่นคลิปในหน้าเครื่อง — ไม่กดก็ไม่ติดต่อ) · ระบบทั้งหมดทำงานได้ถ้าทั้งสองอย่างล่ม เพียงแต่ไม่มีจดหมายออกและคลิปเล่นไม่ได้

4.2 โครงสร้าง repo และ branch

server/	โค้ดฝั่งเซิร์ฟเวอร์ (31 ไฟล์) – app.js ประกอบทุกอย่าง
web/	หน้าเว็บ React (16 ไฟล์) – main.jsx เป็นจุดเริ่ม
shared/	brand.cjs – สูตรคำนวณสีแบรนด์ ไฟล์เดียวที่ทั้งเซิร์ฟเวอร์และเว็บใช้
tests/	เทสต์ API 27 ไฟล์ • tests/ui/ เทสต์เบราว์เซอร์ 16 ไฟล์
docs/	deploy.md • manual/ (คู่มือพนักงาน) • content/ (เนื้อหาพอร์ทัลสมาชิก)
deploy/	entrypoint.sh • bootstrap-env.py • สคริปต์ติดตั้งและสำรอง
data/	ฐานข้อมูลและไฟล์ตอนรันบนเครื่องพัฒนา (บนเครื่องจริงคือ /data)

shared/brand.cjs มีไฟล์เดียวโดยตั้งใจ — เซิร์ฟเวอร์ require ตอนวาดบัตร หน้าเว็บ import ตอนปริ๊นวิวส์ ถ้ามีสองไฟล์ สิบบรรทัดกับสิบบรรทัดจะค่อย ๆ ต่างกันโดยไม่มีใครรู้

4.2.1 branch

branch	ใช้ทำอะไร
release/pilot	รุ่นที่ deploy — งานทั้งหมดรวมที่นี่
main	ปลายทางเมื่อพ้นช่วงทดลอง (ยังไม่ merge)
agent/* , qa/*	branch เก่าของงานเป็นช่วง ๆ เก็บไว้เป็นประวัติ

4.3 ฐานข้อมูล

ไฟล์เดียว SQLite ที่ `DATABASE_PATH` (บนเครื่องจริง `/data/gym.sqlite`) · เปิดแบบ **WAL** และบังคับ `foreign_keys=ON` · ทุกตารางเป็น **STRICT** คือ SQLite ตรวจสอบข้อมูลให้จริง ไม่ให้รับอะไรก็ได้

4.3.1 ตารางที่เก็บข้อมูลส่วนบุคคลของลูกค้า

ตาราง	เก็บข้อมูลลูกค้าอะไร (คอลัมน์สำคัญ)
<code>members</code>	ชื่อ <code>name</code> · เบอร์มือถือ <code>phone</code> (ห้ามซ้ำ) · วันเกิด <code>date_of_birth</code> · ผู้ติดต่อฉุกเฉิน <code>emergency_contact</code> · รหัสสมาชิก <code>member_code</code> · ชื่อไฟล์รูปถ่าย <code>photo_stored_name</code> (ตัวรูปอยู่บนดิสก์ ไม่ได้อยู่ในฐานข้อมูล) · <code>card_version</code> เลขที่บัตรใบปัจจุบัน · <code>welcome_sent_at</code> ส่งบัตรทางอีเมลแล้วเมื่อไหร่ · <code>user_id</code> ชี้ไปบัญชีพอร์ทัลถ้ามีอีเมล
<code>users</code>	อีเมล <code>email</code> · <code>role</code> (<code>admin</code> / <code>staff</code> / <code>member</code>) · <code>password_hash</code> (bcrypt) · <code>name</code> , <code>phone</code> (ใช้เฉพาะเส้นทางสมัครพนักงานเอง ซึ่งปิดอยู่) · <code>status</code> , <code>approval</code>
<code>check_ins</code>	ประวัติการเข้าใช้บริการ — <code>member_id</code> ใครเข้า · <code>checked_in_at</code> เมื่อไหร่ · <code>result</code> ผ่าน/ซ้ำ/ถูกปฏิเสธ · <code>device_label</code> เครื่องไหนสแกน · <code>scanned_by</code> พนักงานคนไหน
<code>orders</code>	ประวัติการรับเงิน — ยอด <code>price_satang_snapshot</code> · วิธีจ่าย <code>payment_method</code> · ชื่อแพ็คเกจตอนขาย (snapshot ไว้ เพราะราคาแพ็คเกจเปลี่ยนทีหลังได้ แต่ใบเสร็จต้องไม่เปลี่ยน)
<code>entitlements</code>	สิทธิ์ที่ลูกค้าถืออยู่ — เริ่ม/หมดอายุเมื่อไหร่ เหลือกี่ครั้ง ถูกยกเลิกหรือยัง
<code>payment_slips</code>	สลิปโอนเงิน — ชื่อไฟล์ ขนาด แฮชไฟล์ เลขอ้างอิง (ตัวรูปอยู่บนดิสก์)
<code>problems_reports</code>	เรื่องที่พนักงานแจ้ง + ชื่อไฟล์ภาพหน้าจอ ซึ่งอาจมีชื่อและรูปสมาชิกที่อยู่บนจอตอนนั้น
<code>audit_logs</code>	ใครทำอะไรกับข้อมูลใคร — <code>actor_id</code> คนทำ · <code>action</code> · <code>entity_id</code> · <code>before_json</code> / <code>after_json</code> ค่าก่อนและหลัง

4.3.2 ตารางที่ไม่ใช่ข้อมูลลูกค้า

กลุ่ม	ตาราง	เก็บอะไร
ข้อมูลยิม	<code>gym_profile</code> , <code>gym_hours</code> , <code>gym_settings</code>	ชื่อ ที่อยู่ เบอร์ เวลาเปิด สี โลโก้ รหัสเชิญ
สินค้า	<code>packages</code>	แพ็คเกจและราคา
อีเมลระบบ	<code>mail_settings</code>	host/port/username · <code>password_sealed</code> รหัสกล่องจดหมายที่เข้ารหัสแล้ว · ผลทดสอบล่าสุด
พอร์ทัลสมาชิก	<code>machines</code> , <code>programs</code> , <code>articles</code> , <code>safety_notices</code>	เนื้อหาวิธีใช้เครื่อง โปรแกรม บทความ ข้อความความปลอดภัย
ความปลอดภัย	<code>sessions</code> , <code>password_setup_tokens</code> , <code>otp_lockouts</code> , <code>rate_limits</code>	เก็บเป็นแซชทั้งหมด ดูข้อ 4.6
ระบบ	<code>schema_migrations</code> , <code>counters</code> , <code>check_in_tokens</code> , <code>otp_challenges</code>	เลข migration · ตัวนับเลขเรื่องแจ้งปัญหา · token บัตรชั่วคราว · ตารางเก่าจากยุค OTP ที่ไม่ได้ใช้แล้ว

รวม **25 ตาราง**

4.3.3 migration

เป็นรายการเลขเรียงใน `server/db.js` ปัจจุบันถึง **018** · รันด้วย `npm run db:migrate` และ `deploy/entrypoint.sh` รันให้อัดโนมัติทุกครั้งที่บูต · ข้ามอันที่ทำไปแล้ว จึงรันซ้ำได้เสมอ

#	ชื่อ	เพิ่มอะไร
001	foundation	สมาชิก ผู้ใช้ session
002	gym_config	ข้อมูลยิม เวลาเปิด แพ้กเกจ
003	orders	คำสั่งซื้อ สิทธิ สลิป
004	checkin	เช็คอินและ token บัตร
005	manual_grant	มอบแพ้กเกจโดยไม่รับเงิน
006	user_status	ระงับบัญชี
007	counter_app	เปลี่ยนเป็นระบบเคาน์เตอร์ — สมาชิกไม่ต้องมีบัญชี พนักงานใช้รหัสผ่าน
008	password_setup	ลิงก์ตั้งรหัสผ่านครั้งเดียว
009–010	gym_branding, appbar_style	สี โลโก้ ชื่อยอบนบัตร
011–012	problem_reports, report_counter	แจ้งปัญหาและเลขเรื่องที่ไม่ย้อนกลับ
013–014	self_signup, verify_and_invite	เส้นทางสมัครเอง (ปิดอยู่) และรหัสเชิญ
015	mail_settings	กล่องจดหมายของยิม + <code>welcome_sent_at</code>
016	member_portal	token อายุ 7 วันของสมาชิก + ช่องวินิจัยอีเมล
017	member_content	เครื่อง โปรแกรม บทความ ข้อความความปลอดภัย
018	audit_survives_delete	สร้าง <code>audit_logs</code> ใหม่ให้ <code>actor_id</code> เป็น <code>ON DELETE SET DEFAULT</code> ซึ้ไปที่บัญชีตัวแทน <code>deleted-user</code> · เพิ่มดัชนี <code>audit_time</code> <code>audit_actor</code> <code>orders_paid</code>

`npm run db:rollback` ต้องตั้ง `ALLOW_DESTRUCTIVE_ROLLBACK=yes` ก่อน และลบข้อมูลที่ migration นั้นสร้าง — สำรองไฟล์ `.sqlite` ก่อนเสมอ

4.4 ที่เก็บไฟล์และอายุการเก็บ

ไฟล์ทุกประเภทไม่ได้อยู่ในฐานข้อมูล — ฐานข้อมูลเก็บแค่ชื่อไฟล์ · ทุกโพลเดอร์อยู่ใต้ volume `/data` ซึ่งเป็นที่เดียวที่รอดจากการ deploy รอบถัดไปและเป็นที่เดียวที่ backup คัดลอก

ประเภท	ตัวแปร	ที่เก็บจริง	ใครเปิดดูได้	อายุการเก็บ
รูปถ่ายสมาชิก	PHOTO_STORAGE_PATH	/data/photos	พนักงานและเจ้าของที่ล็อกอิน · การเปิดดูถูกบันทึก audit	เก็บตลอดอายุสมาชิก ไม่มีการลบอัตโนมัติ
สลิปโอนเงิน	SLIP_STORAGE_PATH	/data/slips	เจ้าของยืม	365 วัน (SLIP_RETENTION_DAYS) ลบด้วย <code>npm run slips:prune</code>
ภาพแจ้งปัญหา	REPORT_STORAGE_PATH	/data/reports	เจ้าของยืมเท่านั้น และบันทึก audit ทุกครั้งที่เปิด	180 วัน (REPORT_RETENTION_DAYS) ลบด้วย <code>npm run reports:prune</code>
โลโก้ยืม	LOGO_STORAGE_PATH	/data/logo	ทุกคน (อยู่บนหน้าเข้าสู่ระบบ)	ไม่ลบ
รูปเครื่องออกกำลังกาย	MACHINE_STORAGE_PATH	/data/machines	ทุกคน (อยู่บนหน้าที่เปิดจาก QR ติดเครื่อง)	ไม่ลบ
ฐานข้อมูล	DATABASE_PATH	/data/gym.sqlite	ไม่มีใครเปิดตรง ๆ ผ่านเว็บได้	ไม่ลบ

รูปถ่ายสมาชิกกับภาพแจ้งปัญหาเป็นข้อมูลส่วนบุคคล ส่วนโลโก้กับรูปเครื่องไม่ใช่ จึงแยกโพลเดอร์กัน — การให้สิทธิ์เข้าโพลเดอร์รูปเครื่องจะได้ไม่พาสิทธิ์เข้าโพลเดอร์รูปหน้าคนไปด้วย

มีเทสต์ `tests/deploy-paths.test.js` ที่อ่านโค้ดเชิร์ฟเวอร์ ว่าใช้ `*_STORAGE_PATH` ตัวไหนบ้าง แล้วบังคับว่าไฟล์ deploy ทั้งสาม (compose/fly/render) ต้องตั้งครบทุกตัวได้ `/data` และ `entrypoint.sh` ต้อง `mkdir` ให้ — โพลเดอร์ที่เพิ่มเดือนหน้าจึงถูกครอบตั้งแต่วันที่เพิ่ม ไม่ต้องมีใครจำได้

4.5 โมเดลสิทธิ์และ endpoint

4.5.1 สามบทบาท

บทบาท	เข้าที่ไหน	ทำอะไรได้
admin (เจ้าของยิม)	/	ทุกอย่าง — ตั้งราคา จัดการบัญชี ตั้งค่ายิมและอีเมล ดูเงิน ดูเรื่องที่แจ้ง แก้เนื้อหาพอร์ทัล
staff (พนักงาน)	/	งานเคาน์เตอร์ — สแกน สมัครสมาชิก แก้ไขข้อมูลสมาชิก รับเงินและมอบแพ็คเกจ ส่งบัตร แจ้งปัญหา · เปิดหน้าเงิน บัญชีผู้ใช้ และเรื่องที่แจ้งไม่ได้
member (ลูกค้า)	/m/login	เห็นเฉพาะ "ช่วยเล่น" ของตัวเอง · เปิด API ของเคาน์เตอร์ไม่ได้เลยสักเส้น และเปิดแอปเคาน์เตอร์ไม่ได้

ทำไมพนักงานแก้ไขข้อมูลสมาชิกได้: อีเมลที่พิมพ์ผิดหนึ่งตัวแปลว่าบัตรสมาชิก (รูป QR ที่ใช้เข้ายิมแทนตัวลูกค้า) ถูกส่งไปกลองจดหมายคนแปลกหน้า ต้องแก้ไขจนขณะลูกค้ายังยืนอยู่ตรงนั้น · สิ่งที่ทำให้ปลอดภัยไม่ใช่สิทธิ์ที่แคบลง แต่คือ **audit** ที่เขียนทุกครั้ง · การระงับสมาชิกยังเป็นของเจ้าของยิม เพราะเป็นการตัดสินใจ ไม่ใช่การแก้คำผิด

4.5.2 endpoint หลัก (รวม 98 เส้น)

กลุ่ม	ตัวอย่าง	ใครเรียกได้
สาธารณะ	GET /api/public/theme · GET /api/public/gym · GET /api/public/packages · GET /api/public/machines · GET /api/public/articles · GET /m/<รหัสเครื่อง>	ทุกคน ไม่ต้องล็อกอิน
เข้าสู่ระบบ	POST /api/auth/login (พนักงาน) · POST /api/auth/member/login (สมาชิก) · POST /api/auth/forgot · POST /api/auth/set-password	ทุกคน (มี rate limit)
ตัวเอง	GET /api/me · POST /api/auth/change-password · POST /api/auth/logout	คนที่ล็อกอิน
แคาน์เตอร์	/api/members/* · /api/check-ins/* · /api/members/:id/grant · /api/members/:id/card	staff + admin
เจ้าของยิม	/api/users/* · /api/packages · /api/gym* · /api/reports · /api/admin/sales · /api/machines* · /api/programs* · /api/articles* · /api/safety · /api/machines/qr-sheet	admin เท่านั้น
ลิงก์เข้าพอร์ทัลที่แคาน์เตอร์	POST /api/members/:id/portal-link	staff + admin · ต้องมีอีเมลของสมาชิกก่อน
รายงานของเจ้าของยิม	GET /api/admin/reports/today · /staff-activity · /sales · /checkins · /members · /issues (ทุกเส้นรับ ?format=csv)	admin เท่านั้น · อ่านอย่างเดียว ไม่เขียน audit
สมาชิก	GET /api/m/me · GET /api/m/home · GET /api/m/programs/:code	member ที่สิทธิ์ยังไม่หมดอายุ

สองด้านของฝั่งสมาชิก: member ตรวจสอบว่าเป็นบัญชีสมาชิกจริง · paidUp ตรวจสอบว่ามีสิทธิ์ที่ยังไม่หมดอายุ ทุกคำขอไม่ใช่ตอนล็อกอิน — สิทธิ์ที่หมดวันอังคารต้องไม่ใช่ได้ต่อจนกว่า session จะหมดวันพุธ · หมดอายุตอบ 402 พร้อมวันที่และชื่อแพ็คเกจ เพื่อให้หน้าจอบอกลูกค้าได้ว่าต้องทำอะไร ไม่ใช่แค่ปฏิเสธ

4.5.3 สิ่งที่ยังบังคับกับทุกคำขอ

- **CORS/CSRF:** คำขอที่เปลี่ยนข้อมูล (ไม่ใช่ GET/HEAD) ต้องมี header `X-Gym-Client` และ `Origin` ตรงตรงกับ `APP_ORIGIN`
 - `Cache-Control: no-store` ทุก response ใต้ `/api`
 - **CSP** จาก helmet · `img-src` เปิดให้ `blob:` (พรีวิวรูปถ่าย) และ `i.ytimg.com` (ภาพนิ่งคลิป) · `frame-src` เปิดให้ `youtube-nocookie.com` เท่านั้น · **ไม่มี inline script ที่ไหนเลย**
 - **rate limit** ต่อ IP และต่อบัญชี ดูตารางข้อ 4.6.4
-

4.6 การเข้ารหัส แสช ลายเซ็น และ token

4.6.1 รหัสผ่าน

bcrypt cost 12 (`bcryptjs`) ประมาณหนึ่งในสี่ในราคาที่ต่อครั้งบนเครื่องนี้ · ใช้ JS ล้วนโดยตั้งใจ ไม่ใช่ native module เพราะ การอัปเดต **Node** แล้ว **native module** คอมไพล์ไม่ผ่าน = ไม่มีใครล็อกอินได้ทั้งยิม ซึ่งเป็นราคาที่แพงกว่าความเร็วที่ได้

การตรวจรหัสผ่านใช้เวลาเท่ากันเสมอ แม้ไม่มีปัญหานั้นอยู่จริง — ถ้าตอบเร็วกว่าเมื่อไม่มีปัญหา คนภายนอกจะไล่หาได้ว่าอีเมลไหนมีอยู่ในระบบ

4.6.2 รหัสกล่องจดหมายของยิม

AES-256-GCM (`server/secret-box.js`) เก็บในคอลัมน์ `mail_settings.password_sealed` รูปแบบ `v1.<iv>.<tag>.<ciphertext>`

คีย์อยู่ใน `.env` ไม่ได้อยู่ในฐานข้อมูล — ไฟล์ `.sqlite` ที่หลุดออกไปเดี่ยว ๆ จึงเป็นไฟล์ที่หลุด ไม่ใช่กล่องจดหมายที่หลุด · ข้อยกเว้นที่ต้องรู้: ชุดสำรองรายวันคัดลอก `.env` ไปไว้ในไฟล์เดียวกับฐานข้อมูล (ดูข้อ 3.5 และ 2.5) ไฟล์สำรองหนึ่งไฟล์จึงมีทั้งข้อมูลและกุญแจ — จำเป็นต่อการกู้คืน แต่แปลว่าไฟล์สำรองต้องถูกเก็บอย่างของลับ

ไม่มี **route** ใดในระบบคืนรหัสนี้กลับมา ไม่ว่าในรูปแบบไหน · หน้าจอเห็นแค่ "ตั้งค่าไว้แล้ว" และไม่อยู่ใน audit log ด้วย

4.6.3 token ทุกชนิดและอายุ

token	ใช้ทำอะไร	อายุ	เก็บอย่างไร
session	ล็อกอินพนักงาน/เจ้าของ	12 ชั่วโมง	ตาราง sessions เก็บ SHA-256 ของ token ไม่ใช่ตัว token · cookie httpOnly sameSite=strict path=/api
session สมาชิก	ล็อกอินพอร์ทัล	12 ชั่วโมง	เหมือนกัน
setup	ลิงก์ตั้งรหัสผ่านที่เจ้าของยิมสร้างให้ หรือออกจากเครื่อง	24 ชั่วโมง	password_setup_tokens เก็บ SHA-256 · ใช้ได้ครั้งเดียว
reset	ลิงก์ "ลืมหั้สผ่าน"	30 นาที	เหมือนกัน — ล้นเพราะคนที่ขอกำลังรออยู่หน้าจอ
verify	ยืนยันอีเมลตอนสมัครเอง	24 ชั่วโมง	เหมือนกัน
member	ลิงก์ตั้งรหัสผ่านที่ไปกับบัตรสมาชิก	7 วัน	เหมือนกัน — ยาวเพราะมาถึงตอนลูกค้ากำลังเดินออกจากยิม
member (ออกที่เคาน์เตอร์)	ปุ่ม "สร้างลิงก์เข้าช่วยเหลือ" บนหน้าบัตร	24 ชั่วโมง	ใช้ครั้งเดียว · เขียน audit member.portal_link_issued ว่าใครออกให้ใคร · จำกัด 10 ครั้ง/ชม./สมาชิก
ลายเซ็นบัตร	QR บนบัตรสมาชิก	ไม่หมดอายุ	ดูข้อ 4.6.5
token บัตรชั่วคราว	ลิงก์ดาวน์โหลดบัตร 7 วัน	7 วัน	check_in_tokens ลบอัตโนมัติหลังหมดอายุ 7 วัน

token หนึ่งใบทำได้อย่างเดียว — ลิงก์ที่ยืนยันอีเมลตั้งรหัสผ่านไม่ได้ และกลับกัน · การขอลิงก์ชนิดหนึ่งไม่ขาลิงก์อีกชนิดที่ยังอยู่ในกล่องจดหมายเดียวกัน

4.6.4 การจำกัดจำนวนครั้ง

เรื่อง	จำกัด
ล็อกอิน (ต่อ IP)	60 ครั้ง / 15 นาที
ล็อกอิน (ต่อบัญชี)	ผิด 5 ครั้ง → ล็อก 15 นาที และบอกจำนวนครั้งที่เหลือ ก่อนถูกล็อก
ลิ้นรหัสผ่าน	20 ครั้ง/IP และ 5 ครั้ง/อีเมล ต่อ 15 นาที
เปลี่ยนรหัสผ่านตัวเอง	10 ครั้ง / 15 นาที
ส่งเมลทดสอบ	5 ครั้ง / 10 นาที
ส่งบัตรทางอีเมล	5 ครั้ง / ชั่วโมง ต่อสมาชิกหนึ่งคน
สร้างลิงก์ตั้งรหัสผ่าน	10 ครั้ง / ชั่วโมง ต่อบัญชีหนึ่งใบ
สแกนเช็किन	300 ครั้ง / 15 นาที ต่อพนักงานหนึ่งคน

จำกัดต่ออีเมลด้วย ไม่ใช่ต่อ IP อย่างเดียว เพราะ IP อย่างเดียวยังปล่อยให้ถล่มกล่องจดหมายคนคนเดียวจากร้อยเครื่องได้

4.6.5 ลายเซ็นบัตรสมาชิก — ข้อที่ต้องระวังที่สุด

QR บนบัตรคือ `gym1.<รหัสสมาชิก>.<เลขที่บัตร>.<ลายเซ็น>` โดยลายเซ็นเป็น **HMAC-SHA256** ด้วยคีย์ `CARD_SIGNING_SECRET` · การตรวจใช้ `timingSafeEqual` เพื่อไม่ให้หาลายเซ็นถูกได้ด้วยการจับเวลาคำตอบ

บัตรไม่มีวันหมดอายุ และไม่มีรายการบัตรที่ถูกยกเลิก — การยกเลิกบัตรทำโดยนับเลขที่บัตรขึ้นหนึ่ง ใบเท่าจึงมีลายเซ็นที่ไม่ตรงกับเลขปัจจุบันทันที ไม่ต้องไปตามเก็บบัตรที่ส่งออกไปแล้ว

 **CARD_SIGNING_SECRET ห้ามเปลี่ยนเด็ดขาด** · เปลี่ยนเมื่อไหร่ บัตรทุกใบที่ลูกค้าถืออยู่สแกนไม่ติดพร้อมกันทั้งยิม และทุกคนถูกออกจากระบบ · ภูมิภาค ต้องออกบัตรใหม่ให้สมาชิกทุกคน · คีย์นี้ต้องยกไปด้วยเมื่อย้ายเครื่อง

4.6.6 การป้องกันการไล่หาว่าใครมีบัญชี

สามหน้าจอตบประโยคเดียวกันทุกกรณี: เข้าสู่ระบบ · ลืมรหัสผ่าน · เข้าสู่ระบบสมาชิก

- ลืมรหัสผ่านด้วยอีเมลที่ไม่มีบัญชี — ไม่มีจดหมายออกไปเลยสักฉบับ และใช้เวลาเท่ากับกรณีที่มีบัญชี (ระบบสร้าง token แล้วทิ้ง เพื่อให้เวลาตอบเท่ากัน)
- หน้าเข้าสู่ระบบสมาชิก ตอบเหมือนเดิมแม้อีเมลนั้นเป็นของพนักงานจริง ๆ — ไม่จันหน้านี้กลายเป็นรายการว่าอีเมลไหนทำงานที่ยิม บนหน้าที่ไม่ต้องล็อกอิน
- หน้าลืมรหัสผ่านฝังสาธารณะไม่เคยแสดงถึงบนจอ ไม่ว่าใครกด

4.7 การส่งอีเมล

ส่งผ่าน **SMTP** ของกล่อง **Office 365** ของยิมเอง — `smtp.office365.com` พอร์ต **587** STARTTLS · ไม่มีผู้ให้บริการภายนอก ไม่มีค่าใช้จ่ายเพิ่ม และผู้รับเห็นอีเมลที่เขารู้จัก

ค่าตั้งอยู่ในฐานข้อมูล ไม่ใช่ `.env` เพราะคนที่รู้รหัสกล่องจดหมายคือเจ้าของยิม และหน้าจอดีวที่เขาเข้าถึงได้คือแอป · กรอกที่ ตั้งค่ายิม → แท็บ "อีเมลของระบบ" (เฉพาะเจ้าของยิม)

- `requireTLS` เปิดอยู่ — ถ้าเซิร์ฟเวอร์ไม่ยอมเข้ารหัส ระบบจัดการเชื่อมต่อทั้งโดยไม่ส่งรหัสผ่านออกไป ส่งไม่สำเร็จดีกว่าส่งรหัสกล่องจดหมายแบบอ่านได้
- คิวส่งเว้นระยะ ~2 วินาทีต่อฉบับ เพราะ Office 365 รับราว 30 ฉบับ/นาที/กล่อง
- **log** บันทึกแค่ปลายทางและหัวเรื่อง ไม่เคยบันทึกเนื้อหาจดหมาย เพราะในนั้นมีลิงก์ที่ใช้ได้จริง — ลิงก์ใน log คือลิงก์ที่คนอ่าน log ใช้ได้
- ยังไม่ได้ตั้งค่า = ระบบทำงานปกติทุกอย่าง แต่ไม่มีจดหมายออก

จดหมาย 5 ฉบับ (`server/emails/`) สร้างจากไฟล์ต้นทางเดียว `mkmail.cjs` ทั้ง HTML และข้อความล้วน — เพื่อไม่ให้สองฉบับพูดคนละเรื่อง · ทุกฉบับไม่มีรูป ไม่มีสคริปต์ ไม่มี **List-Unsubscribe** (เป็นจดหมายธุรกรรม) และสื่่อมมาจากสีของยิมเอง

ความล้มเหลวถูกแปลเป็น 4 กลุ่มที่เจ้าของยิมทำตามได้: ยังไม่เปิด **Authenticated SMTP** · รหัสผ่านใช้ไม่ได้ (ต้องใช้ **App password**) · เซิร์ฟเวอร์ไม่รองรับการเข้ารหัส · ต่อไม่ได้ — บวกรณีไม่รู้จึกที่ยังบอกว่าให้กดแจ้งปัญหา · สองกลุ่มแรกมาจาก Microsoft เป็น 535 เหมือนกันทั้งคู่ แต่วิธีแก้คนละเรื่อง การอ่านรวมกันคือการส่งเจ้าของยิมไปพิมพ์รหัสผ่านใหม่ทั้งที่ไม่ใช่ปัญหา

4.8 เทสต์และตัวกันหลุด

ชุด	จำนวน	สั่งรัน
เทสต์ API และตรรกะ	276 ข้อ (34 ไฟล์)	<code>npm test</code>
เทสต์เบร้าวเซอร์จริง	119 ข้อ (20 ไฟล์)	<code>npm run test:ui</code>

เทสต์เบร้าวเซอร์รัน Playwright บนเชิร์ฟเวอร์ทดสอบสามตัว (ปกติ · โหมดทดลอง · เปิดสมัครเอง) ฐานข้อมูลในหน่วยความจำ ไม่แตะเครื่องจริง

ตัวกันหลุดที่อ่านโค้ดจริง — ไม่ใช่รายการที่ต้องมานั่งอัปเดตเอง:

เทสต์	กันอะไร
<code>deploy-paths.test.js</code>	โพลเดอร์เก็บไฟล์ใหม่ที่ลื้มตั้งใน <code>compose/fly/render</code> — อ่านจากโค้ดเชิร์ฟเวอร์
<code>dockerfile-copies.test.js</code>	ไฟล์ที่ Dockerfile คัดลอกแต่ <code>.dockerignore</code> ตัดออก
<code>mail-settings.test.js</code> (ข้อสุดท้าย)	กึ่งความล้มเหลวที่เพิ่มในเชิร์ฟเวอร์แต่ลื้มเพิ่มในหน้าจอ — เทียบซอร์สสองไฟล์
<code>09-tokens.spec.js</code>	<code>var(--xxx)</code> ที่ไม่มีนิยาม กวาดทุกหน้าจอ
<code>07-readable.spec.js</code>	ตัวอักษรที่คอนทราสต์ต่ำกว่า 4.5:1 และเป้าแตะเล็กกว่า 44px
<code>member-portal.test.js</code>	session สมาชิกยี้ง 18 route ของเคาน์เตอร์ ต้องได้ 403/404 ทุกอัน
<code>deploy-cron.test.js</code>	script <code>*:prune</code> ใน <code>package.json</code> ที่ไม่มีบรรทัด cron ในตัวติดตั้ง — อ่าน <code>package.json</code> เทียบกับ <code>deploy/bootstrap.sh</code>

ข้อสุดท้ายคือตัวที่จับได้ว่า session สมาชิกเปิดหน้าคอนโซลพนักงานได้ ก่อนขึ้นเครื่องจริง

4.9 รันบนเครื่องพัฒนา

ต้องมี **Node 24** เท่านั้น ไม่ต้องมีฐานข้อมูลหรือ Docker

```
npm ci
cp .env.example .env          # แล้วเติม CARD_SIGNING_SECRET กับ SETTINGS_ENC_KEY
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"  # ใช้สร้างคีย์ทั้งสอง

npm run db:migrate            # สร้างตารางทั้งหมด
npm run db:seed               # ข้อมูลยืมเริ่มต้นและแพ็กเกจว่าง
npm run content:import        # เนื้อหาพอร์ทัลสมาชิก
npm run db:admin              # สร้างบัญชีผู้ดูแลจาก ADMIN_EMAIL/ADMIN_PASSWORD ใน .env

npm run dev                   # หน้าเว็บ (Vite) ที่ http://127.0.0.1:5173
npm start                     # เซิร์ฟเวอร์ที่ http://127.0.0.1:3000
```

ก่อนส่งงานทุกครั้ง:

```
npm test                      # 276 ข้อ
npm run build                  # ต้องผ่านก่อนรันเทสต์เบร้าวเซอร์
node tests/free-port.js && npx playwright test  # 119 ข้อ
```

คำสั่งอื่นที่มี: `npm run admin:set-password-link -- <email>` (ทางเข้าสำรองเมื่อยังส่งเมลไม่ได้) · `npm run env:set -- --list` (ดูว่ามีคีย์อะไรบ้างโดยไม่แสดงค่า) · `npm run mail:build` (สร้างเทมเพลตจดหมายใหม่จาก `mkmail.cjs`)

4.10 สิ่งที่ต้องรู้ก่อนแต่ละระบบนี้

สามข้อที่ราคาแพงที่สุดถ้าพลาด:

1. `CARD_SIGNING_SECRET` ห้ามเปลี่ยน — บัตรทุกใบตายพร้อมกัน ทุไม่ได้
2. `SETTINGS_ENC_KEY` ต้องยกไปด้วยตอนย้ายเครื่อง — ไม่ยกไป เจ้าของยืมต้องกรอกรหัสกล่องจดหมายใหม่
หนึ่งครั้ง (ระบบบอกสถานะตรง ไม่ได้เจียบ)
3. `SELF_SIGNUP` ต้องไม่เป็น 1 — ตั้งพลาดเมื่อไหร่ ประตูหน้าเปิดให้คนทั้งอินเทอร์เน็ตส่งค่าขอเข้าคิวอนุมัติทันที

ทั้งสามข้ออยู่ในเช็คลิสต์หลัง deploy ของ `docs/deploy.md` พร้อมคำสั่งตรวจ

4.11 บทเรียนจากการจับภาพหน้าจอ (BUG-14 ถึง BUG-17)

ปุ่ม "แจ้งปัญหา" จับภาพหน้าจอก่อนเปิดกล่องเสมอ · เส้นทางนี้พึงสืแบบในรอบเดียว ทุกแบบเจอจากการทดลองใช้จริงและแก้ไปแล้ว

#	อาการ	รากของปัญหา	สิ่งที่ทำ
14	จอสแกนค้างที่ "กำลังจับภาพหน้าจอ..." เป็นนาที่ กดอะไรไม่ได้	ลูปอ่าน QR กับตัวจับภาพแย่ง main thread กันจนภาพไม่เสร็จ	หยุดถอดรหัส QRชั่วคราวระหว่างจับภาพ (กล่องยังเปิด) · timeout 8 วินาทีวางไว้ในฟังก์ชัน เพื่อให้ <code>finally</code> คืน <code><video></code> เสมอ · เพิ่มปุ่ม "ยกเลิก" ในแผง · แต่ละครั้งที่กดมีเลขกำกับ กล่องที่ถูกยกเลิกแล้วจึงแจ้งเตือนขึ้นทีหลังไม่ได้
15	กล่องเล็กอ่านบัตรถาวร ถ้าไฟล์ของตัวจับภาพโหลดไม่สำเร็จ	ธงหยุดอ่านถูกตั้งหลังโหลดโมดูล พอโหลดฟังก์จึงไม่มีใครคืนค่า	ครอบด้วย <code>withScanningPaused</code> ตั้งแต่บรรทัดแรก ธงจึงถูกคืนทุกเส้นทางรวมทั้งตอนโหลดไม่สำเร็จ
16	จับภาพ ครั้งแรกของหน้า ช้ากว่าครั้งถัดไปมากจนเกือบชน timeout	โมดูลจับภาพถูกโหลดตอนกดปุ่ม	โหลดล่วงหน้าตอนว่าง หลังคนล็อกอินและแอปเคาน์เตอร์ขึ้นแล้ว (<code>requestIdleCallback</code> + ตัวสำรอง) · ไม่โหลดในพอร์ทัลของสมาชิก ซึ่งไม่มีปุ่มแจ้งปัญหาให้ใช้
17	ทุกภาพที่ส่งมามีแผง "กำลังจับภาพหน้าจอ..." บังสิ่งที่กำลังจะรายงาน	แผงอยู่บนจอขณะชัตเตอร์เปิด	ตัดโน้ตที่ติดป้าย <code>data-capture-hide</code> ออกจากภาพ

ผลงานเครื่องจริงหลังแก้ครบ จับภาพได้ทุกครั้ง ครั้งแรก 1.82 วินาที ครั้งถัดไป 1.81 วินาที และภาพไม่มีแผงติดไปด้วย

ถ้าต้องแตะส่วนนี้อีก กฎสามข้อ: ธงหยุดอ่านต้องถูกคืนใน `finally` เสมอ · `timeout` ต้องอยู่ในฟังก์ชันเดียวกับที่คืนธง ไม่ใช่ครอบจากข้างนอก · ทุกอย่างที่ว่าดทับจระหว่งจับภาพต้องมี `data-capture-hide`

4.12 รายงานของเจ้าของยิม และการลบบัญชี (รุ่น b8a9d96)

4.12.1 endpoint รายงาน

`server/admin-reports.js` · หักเส้นใต้ `/api/admin/reports/*` เปิดเฉพาะ `admin` · ทุกเส้นเป็นการอ่านล้วน และ **ไม่เขียนแถว `audit`** โดยตั้งใจ — บันทึกที่เต็มไปด้วย "มีคนเปิดดูตาราง" คือบันทึกที่ไม่มีใครอ่าน

กติกา	ค่า
เขตเวลาของทุกวันที่	<code>date(ts/1000, 'unixepoch', '+7 hours')</code> — วันตามเวลากรุงเทพ
ช่วงที่ขอได้	ไม่เกิน 366 วัน ต่อหนึ่งคำขอ
เพดานแถวที่ส่งกลับเป็น JSON	5,000 แถว พร้อม <code>truncated: true</code> และ <code>total</code> เต็ม
เพดานของ CSV	ไม่มี — <code>?format=csv</code> ส่งครบทุกแถวเสมอ
รูปแบบ CSV	UTF-8 พร้อม BOM · CRLF · เวลา ISO <code>+07:00</code> คู่กับวันที่ไทย พ.ศ. · เงินเป็นตัวเลขทศนิยม 2 ตำแหน่ง · เบอร์โทรมีขีด
ยอดขายนับจาก	<code>orders.reviewed_at</code> เฉพาะ <code>status='paid'</code> AND <code>manual_grant=0</code>
ประวัติการสแกน อ่านจาก	<code>check_ins</code> ทั้งตาราง ไม่ใช่ <code>audit_logs</code> ซึ่งมีแต่ <code>checkin.allowed</code>

สามกับดักที่เขียนไว้ในหัวไฟล์ และห้ามลืมเมื่อแก้ต่อ

- แถว `checkin.allowed` ใน `audit_logs` เก็บรหัสแถวเช็คอินไว้ใน `entity_id` ทั้งที่ `entity_type` ยังเป็น `'member'` — join ตรง ๆ กับ `members` จะไม่เจออะไรเลย
- `before_json / after_json` **ไม่ออกจากไฟล์นี้** สองคอลัมน์นั้นมีชื่อ เบอร์โทร และอีเมลของสมาชิก · รายงานบอกได้แค่ว่า**ช่องไหนถูกแก้** ("แก้: เบอร์โทร, อีเมล") ไม่บอกว่าแก้เป็นอะไร
- `check_ins.failure_reason` เป็นข้อความไทยอิสระ **ไม่ใช่ enum** — จัดกลุ่มด้วยคำสำคัญเท่านั้น การ `GROUP BY` ตรงตัวจะพังวันที่มีคนแก้คำผิดในข้อความ

`tests/report-actions.test.js` อ่าน `server/*.js` หา action ที่เขียนลง `audit_logs` จริง (ปัจจุบัน **42** ตัว) แล้ว บังคับว่าทุกตัวต้องมีค่าแปลไทยในตาราง `ACTIONS` — action ที่เพิ่มเติมหน้าแล้วล้มแปล จะทำให้เทสต์แดง แทนที่จะไปโผล่เป็นรหัสดิบบนหน้าจอเจ้าของยิม

4.12.2 `user.login` / `user.logout`

เขียนลง `audit_logs` เฉพาะตอนล็อกอินสำเร็จ · `after_json` มีแค่ `{ role }` · ไม่เก็บหมายเลข IP และไม่เก็บ **user agent** · การล็อกอินที่ล้มเหลวไม่ถูกบันทึกเลย (การกั้นคนสมัครห้ใช้ rate limit และการล็อกบัญชีอยู่แล้ว) · แกว `user.logout` มีเฉพาะตอนกดออกเอง session ที่หมดอายุไปเงียบ ๆ ไม่ทิ้งแถว

เจตนาเชิงนโยบาย: รายงานนี้ตอบว่าใครเข้าเวร ไม่ได้ตอบว่าเขายืนอยู่ที่ไหน · ถ้าวันหน้าตัดสินใจเก็บ IP ต้องเก็บที่ 2 (ประเภทข้อมูลส่วนบุคคล) และกำหนดอายุการเก็บพร้อมกัน · มีเทสต์ที่ยิง regex หา IPv4 และชื่อเบราว์เซอร์ในแถวที่เขียนจริง เพื่อกันของหลุดในอนาคต

4.12.3 migration 018 และสิ่งที่มันยังทำไม่ได้

018_audit_survives_delete สร้างตาราง `audit_logs` ขึ้นใหม่ (SQLite แก่ foreign key ในที่เดิมไม่ได้) แล้วย้ายข้อมูลเดิมมาทั้งหมด · `actor_id` มี DEFAULT 'deleted-user' และ ON DELETE SET DEFAULT · บัญชีตัวแทน `deleted-user` ถูกสร้างพร้อมกัน: ไม่มีรหัสผ่าน สถานะ `suspended` ล็อกอินไม่ได้ และถูกซ่อนจาก `GET /api/users` กับดรอปลาวนัต์ตรวจสอบของรายงาน — มันเป็นปลายทางของ foreign key ไม่ใช่คนที่ทำงานที่นี่ · รายงานแปลงเป็นข้อความ "บัญชีที่ถูกลบ"

ข้อจำกัดที่ยังอยู่ และต้องรู้ก่อนวางแผนลบบัญชี migration 018 ครอบคลุมเฉพาะ `audit_logs` ตารางอื่นยังอ้าง `users` แบบไม่มี ON DELETE :

```
password_setup_tokens.user_id · check_ins.scanned_by · orders.reviewed_by ·
problem_reports.reported_by · members.user_id
```

ผลจริงในวันนี้คือยังลบบัญชีไหนไม่ได้เลย เพราะทุกบัญชีที่สร้างผ่านหน้า "บัญชีผู้ใช้" จะมีแถวใน `password_setup_tokens` เสมอ (ระบบออกลิงก์ตั้งรหัสให้พร้อมกับการสร้างบัญชี) — 018 จึงเป็นเกราะที่เตรียมไว้ล่วงหน้า ยังไม่ใช่ของที่ทำงานอยู่ และตัวที่ติดจริงตัวแรกคือ `password_setup_tokens` ถ้าจะทำ migration 019 ให้เริ่มที่ตารางนั้น

ทางที่ใช้อยู่และปลอดภัยกว่ายังเป็น ระวังบัญชี ซึ่งปิดการเข้าใช้ทันทีและเก็บประวัติไว้ครบ (ดูข้อ 2.4)

4.12.4 endpoint บัญชีผู้ใช้ที่เปลี่ยนไป

เส้น	เปลี่ยนอะไร
POST <code>/api/users</code>	ถ้าไม่ส่ง <code>password</code> มา ระบบออกลิงก์ตั้งรหัส 24 ชั่วโมงให้พร้อมกับบัญชี และคืนมาใน <code>setup_link</code> · เขียน audit <code>user.create</code> และ <code>user.password_link_issued</code>
PUT <code>/api/users/:id/role</code>	ปฏิเสธ 409 เมื่อแกสิทธิ์ของตัวเอง — "เปลี่ยนสิทธิ์ของตัวเองไม่ได้ ให้ผู้ดูแลระบบอีกคนเป็นคนเปลี่ยนให้" · ด้านกันผู้ดูแลระบบเหลือศูนย์คนยังอยู่เหมือนเดิม
GET <code>/api/users</code>	ไม่คืนบัญชีตัวแทน <code>deleted-user</code> และไม่นับเป็นผู้ดูแลระบบ

ภาคผนวก ก · คำศัพท์ รายการตรวจ และการติดต่อ

ก.1 คำศัพท์ที่โผล่บ่อยในเล่มนี้

คำ	แปลว่าอะไรในระบบนี้
VPS	เครื่องเซิร์ฟเวอร์ที่ยืมเอาไว้ ทำงานอยู่ตลอดเวลา ระบบทั้งหมดอยู่บนเครื่องนี้
Docker / คอนเทนเนอร์	วิธีห่อโปรแกรมให้รันเหมือนกันทุกเครื่อง ระบบนี้มีสองห่อ: ตัวแอป และตัวรับ HTTPS
Caddy	ตัวรับคำขอจากอินเทอร์เน็ต ทำ HTTPS ให้เอง แล้วส่งต่อให้แอป
volume	ที่เก็บไฟล์ถาวรของคอนเทนเนอร์ · ของที่อยู่นอก volume จะหายตอนอัปเดตรุ่น
/data	volume ที่เก็บของจริงทั้งหมด: ฐานข้อมูล รูปถ่าย สลิป โลโก้ ภาพแจ้งปัญหา รูปเครื่อง
.env	ไฟล์ค่าตั้งค่าและค่าลับของเครื่อง อ่านได้เฉพาะ root
SQLite	ฐานข้อมูลที่เป็น "ไฟล์เดียว" ไม่ต้องมีเซิร์ฟเวอร์ฐานข้อมูลแยก
migration	ขั้นตอนอัปเดตโครงสร้างฐานข้อมูลเป็นลำดับเลข ระบบรันให้เองทุกครั้งที่บูต
cron	ตารางงานตามเวลาของเครื่อง เช่น สำรองข้อมูลดี 2:17
bcrypt	วิธีเก็บรหัสผ่านแบบที่อ่านกลับไม่ได้ ใช้ตรวจได้อย่างเดียวว่าพิมพ์ถูกไหม
AES-256-GCM	การเข้ารหัสที่ใช้เก็บรหัสกล่องจดหมายของยิม ต้องมีกุญแจจึงถอดได้
HMAC / ลายเซ็น	รหัสสั้น ๆ ที่พิสูจน์ว่า QR บนบัตรออกจากระบบนี้จริง ปลอมไม่ได้ถ้าไม่มีกุญแจ
token	บัตรผ่านชั่วคราวในรูปข้อความยาว ๆ เช่น ลิงก์ตั้งรหัสผ่าน มีอายุและใช้ได้ครั้งเดียว
audit	บันทึกว่าใครทำอะไรกับข้อมูลใครเมื่อไหร่
rate limit	การจำกัดจำนวนครั้งต่อช่วงเวลา กันคนสุมรหัสผ่านและกันเมลกล่ม
QA / GO	รอบตรวจก่อนติดตั้ง และคำนวณมิติให้ติดตั้งได้

ก.2 รายการตรวจตามรอบ

ทุกสัปดาห์ — เจ้าของยิมหรือผู้ดูแล (5 นาที)

1. ก๊อปปี้ไฟล์สำรองล่าสุดออกนอกเครื่องด้วย `scp` (ข้อ 3.5) แล้วดูว่าไฟล์ที่ได้ลงวันที่ล่าสุดจริง

ทุกเดือน — ผู้ดูแลเครื่อง (15 นาที)

1. `df -h /` พื้นที่ดิสก์ — เกิน 70% ให้ลดอิมเมจเก่าและตรวจขนาด `/var/backups/gym`
2. `tail /var/log/gym-backup.log /var/log/gym-prune.log` — ต้องไม่มี error และต้องเห็นบรรทัดของวันล่าสุด
3. ในรับรอง HTTPS ขยับวันหมดอายุไปข้างหน้า (Caddy ต่อให้เอง)
4. `apt-get update && apt-get -y upgrade` แล้วดู `/var/run/reboot-required`
5. วันหมดอายุ VPS และโดเมนในบัญชี Hostinger — ควรเปิดต่ออายุอัตโนมัติ

ทุกไตรมาส — ผู้ดูแลเครื่อง

1. **ซ้อมกู้คืนหนึ่งครั้ง** จากไฟล์สำรองล่าสุดลงเครื่องทดสอบ (ข้อ 3.5) — ไฟล์สำรองที่ไม่เคยกู้ คือไฟล์สำรองที่ยังไม่รู้ว่าจะใช้ได้
2. ตรวจว่าบัญชีพนักงานในระบบยังตรงกับคนที่ทำงานอยู่จริง ใครลาออกแล้วให้ระงับบัญชี

ทุกครั้งที่อัปเดตระบบ ทำตามเช็กลิสต์ 6 ข้อในข้อ 3.7 และสำรองข้อมูลก่อนเสมอ

ก.3 ใครติดต่อเรื่องอะไร

เรื่อง	ติดต่อ
ระบบเปิดไม่ได้ · เว็บล่ม · ใบบริบ รอมหมดอายุ	ผู้ดูแลเครื่อง (ทีมติดตั้ง)
อีเมลของระบบส่งไม่ออก	เจ้าของยิมตรวจหา "อีเมลของระบบ" ตามหน้าวินิจฉัยก่อน · ถ้าเป็นเรื่องนโยบายของ Microsoft 365 ติดต่อผู้ดูแล Microsoft 365 ของยิม
ลูกค้าขอให้ลบข้อมูล	เจ้าของยิมรับเรื่อง แล้วส่งให้ผู้ดูแลเครื่องทำตามข้อ 2.4
พนักงานลืมรหัสผ่าน	เจ้าของยิมสร้างลิงก์ตั้งรหัสผ่านให้จากหน้า "ผู้ใช้และสิทธิ์" (คู่มือพนักงาน ข้อ 2.2)
บัตรสมาชิกหลุดไปถึงคนอื่น	เจ้าของยิมกด "ออกบัตรใหม่" (คู่มือพนักงาน ข้อ 2.6)
อยากได้พีเจอาร์ใหม่ / เจอบั๊ก	กด "แจ้งปัญหา" ในแอป แล้วแจ้งทีมพัฒนา

ก.4 เปลี่ยนอะไรในรุ่น 1.3

- เขียนจากรุ่น `b8a9d96` ที่ผ่าน QA แล้วและกำลังจะขึ้นเครื่องจริง (เครื่องจริงขณะเขียนยังเป็น `856846c`) · เทสต์ **276** ข้อ และเทสต์เบราร์เซอร์ **119** ข้อ
- บทที่ 4 เพิ่มข้อ **4.12** — endpoint รายงานหกเส้นใต้ `/api/admin/reports/*` (อ่านอย่างเดียว ไม่เขียน audit) · เพดาน **5,000** แถวของ **JSON** ส่วน **CSV** ไม่มีเพดาน · กติกา CSV (UTF-8 พร้อม BOM · CRLF · เวลา ISO คู่กับวันที่ไทย · เงินทศนิยม 2 ตำแหน่ง · เบอร์โทรมีขีด) · `user.login / user.logout` ที่ไม่เก็บ **IP** และไม่เก็บชื่อเบราร์เซอร์ · **migration 018** และข้อจำกัดที่ยังเหลือ · endpoint บัญชีผู้ใช้ที่เปลี่ยนไป
- บทที่ 4 อัปเดตตาราง migration เป็น **018** ตาราง endpoint เป็น **98** เส้น และเพิ่มแถวรายงานของเจ้าของยิม
- บทที่ 2 แถว audit เพิ่มการเข้า/ออกจากระบบ และระบุว่า เจ้าของยิมอ่านเองได้จากแท็บ "รายงาน" แล้ว (เดิมต้องให้ผู้ดูแลเครื่องดึงจากฐานข้อมูล) · ข้อ **2.4** เขียนใหม่ ตาม migration 018: แถว audit อยู่รอดเมื่อลบบัญชี แต่ ในทางปฏิบัติยังลบบัญชีไหนไม่ได้ เพราะ `password_setup_tokens`
- `.env` ไม่เปลี่ยนจากรุ่น **1.2** — คียยังคง 15 ตัว รอบนี้ไม่มีคีย์ใหม่และไม่มีคีย์ที่ถูกลบ
- ภาคผนวก ก.6 อัปเดตสถานะและรายการงานถัดไปให้ตรงปัจจุบัน

ก.4ก เปลี่ยนอะไรในรุ่น 1.2

- เครื่องจริงติดตั้งรุ่น 856846c แล้ว (ต่อจาก 115fb17) · เทสต์ **247** ข้อ และเทสต์เบราว์เซอร์ **111** ข้อ
- บทที่ 4 เพิ่มหน้าจอ "เครื่องและโปรแกรม" ฝั่งเจ้าของยิมในตาราง endpoint · เพิ่ม ลิงก์เข้าพอร์ทัลที่เคาน์เตอร์สร้างได้ (24 ชั่วโมง ใช้ครั้งเดียว มี audit) ในตาราง token · และเพิ่ม ข้อ 4.11 บทเรียนจากการจับภาพหน้าจอ (BUG-14 ถึง 17) พร้อมกฎสามข้อสำหรับคนที่จะแตะส่วนนี้ต่อ
- บทที่ 2 เพิ่มการกระทำที่ถูกบันทึกใหม่ (สร้างลิงก์เข้าช่วยเล่น · แก้เนื้อหาเครื่อง/โปรแกรม · อนุมัติข้อความความปลอดภัย) และ ข้อจำกัดของ audit_logs.actor_id ที่ทำให้ลบบัญชีซึ่งเคยทำอะไรไว้ไม่ได้
- บทที่ 3 อัปเดตข้อ 3.11 เป็นรุ่นปัจจุบัน · .env ไม่เปลี่ยนจากรุ่น 1.1
- ภาคผนวกเพิ่ม ก.6 สถานะการพัฒนาและงานถัดไป

ก.4ข เปลี่ยนอะไรในรุ่น 1.1

- เครื่องจริงติดตั้งรุ่น `115fb17` แล้ว ทั้งเล่มจึงเป็นคำอธิบายของเครื่องที่ให้บริการอยู่จริง ไม่ใช่รุ่นที่รอขึ้น
- บทที่ 2 เพิ่มข้อ 2.4 (ต่อ) — อายุการเก็บข้อมูลต้องนับสามชั้น: ในระบบ + ไม่เกิน 7 วันในชุดสำรอง + ตามรอบของสำเนาออกเครื่อง พร้อมประโยคที่ใช้ตอบได้จริงเมื่อมีคนถาม
- บทที่ 3 เพิ่มสองกับดักของ `.env` ที่เจอบนเครื่องจริง (ต้องแก้ก่อน `up.sh` · ห้ามรัน `env:set` ผ่าน `docker compose exec`) · คีย์ `SMTP_*` และ `MAIL_FROM` 6 ตัวถูกลบออกจากเครื่องแล้ว · เพิ่มข้อ 3.11 บัญชีและรุ่นที่ติดตั้งอยู่
- บทที่ 4 อัปเดตเป็นรุ่นที่ติดตั้ง: เทสต์ 236 ข้อ และเทสต์เบร้าวเซอร์ 95 ข้อ

ก.6 สถานะการพัฒนาและงานถัดไป

สถานะ ณ 20 กันยายน 2569 เครื่องจริงให้บริการอยู่บนรุ่น `856846c` · รุ่น `b8a9d96` (รายงานสำหรับเจ้าของยิม และหน้าจัดการบัญชีผู้ใช้) ผ่าน QA ทั้งฝั่ง API และหน้าจอแล้ว รอขึ้นเครื่อง · งานเคาน์เตอร์ พอร์ตลสมาชิก อีเมลของระบบ หน้าจอเนื้อหา และรายงาน ครบทั้งหมด · เหลือฝั่งเจ้าของยิมสองเรื่องก่อนใช้เต็มรูปแบบ: กรอกรหัสกล่องจดหมายแล้วกดส่งเมลทดสอบ และตั้งราคาแพ็คเกจที่ยังเป็นร่าง · บัญชีของยิมทั้งสองบัญชีเป็นผู้ดูแลระบบตั้งแต่ 20 ก.ย. 2569

งานถัดไปที่บันทึกไว้แล้ว (ยังไม่มีกำหนด เรียงตามที่กระทบผู้ใช้จริงมากที่สุดก่อน)

งาน	ทำไมต้องมี
migration 019 — ให้ตารางที่เหลืออ้าง <code>users</code> แบบมีกฎ "ลบตาม" เริ่มที่ <code>password_setup_tokens</code>	migration 018 ทำให้แถว audit อยู่รอดแล้ว แต่ ยังลบบัญชีไหนไม่ได้จริง เพราะตารางอื่นค้างอยู่ (ข้อ 4.12.3)
ลบสมาชิกถาวรจากในแอป พร้อมขั้นตอนยืนยันยันสองชั้น	คำขอลบข้อมูลของลูกค้าต้องทำได้จบในแอป ไม่ต้องให้ผู้ดูแลเครื่องลงมือในฐานข้อมูล (ข้อ 2.4)
อายุการเก็บของ <code>access.log</code> ฝั่ง Caddy	เป็นข้อมูลส่วนบุคคล (IP) ที่ยังหมุนตามขนาดไฟล์ ไม่ใช่ตามอายุ (ข้อ 2.5)
เข้าสู่ระบบด้วย Google	ปุ่มบนหน้าแรกยังปิดอยู่ · ลดการลืมหืมผ่านของพนักงาน
เทสต์เทียบเอกสารกับโค้ด	กันเอกสารกับระบบเดินคนละทาง แบบเดียวกับ <code>deploy-paths</code> และ <code>deploy-cron</code> ที่อ่านขอร์สจริง
CI ที่ build จริงทุกครั้ง	บ๊ัก build สองครั้งที่ผ่านมาไม่มีเทสต์ไหนจับได้ เพราะไม่มีชุดไหนรัน <code>docker build</code>
merge <code>release/pilot</code> เข้า <code>main</code>	ตอนนี้งานทั้งหมดอยู่บน branch ทดลอง
เปิดรับเงินผ่านพร้อมเพย์	ต้องมีเลขบัญชีจากเจ้าของยิมก่อน · จนกว่าจะเปิด แถบ "โหมดทดลอง" ยังอยู่และต้องอยู่

ก.5 ขอบเขตของเอกสารฉบับนี้

- เขียนจากโค้ดจริงของรุ่น `b8a9d96` ซึ่ง ผ่าน QA แล้วและกำลังจะขึ้นเครื่อง · ทุกอย่างในเล่มเป็นของที่ติดตั้งอยู่แล้วบนเครื่องจริง ยกเว้นสามเรื่องของรอบนี้ ที่จะเป็นจริงเมื่อรุ่นนี้ขึ้นเครื่อง: แท็บ "รายงาน" · แท็บ "บัญชีผู้ใช้" · migration 018 (ดูข้อ 4.12)
- ตัวเลขทั้งหมด (อายุการเก็บ · เวลา cron · อายุ token · จำนวนเทสต์) ยกมาจากโค้ดและไฟล์ตั้งค่าจริง ไม่ได้ประมาณ
- สิ่งที่ระบบยังไม่มี ย้ายไปอยู่ในภาคผนวก ก.6 พร้อมเหตุผลของแต่ละข้อ · หน้าจอฝั่งเจ้าของยิมสำหรับยืนยันเนื้อหาพอร์ทและอัปโหลดรูปเครื่อง มีแล้วในรุ่นนี้ (ดูคู่มือพนักงาน ข้อ 2.10)
- เล่มนี้อัปเดตเมื่อโครงสร้างเครื่อง ข้อมูล หรือรอบการสำรองเปลี่ยน — ไม่ต้องอัปเดตทุกครั้งที่ปล่อยรุ่นใหม่ของแอป